King Saud University
College of Computer and Information Sciences
Computer Science Department

المُرَتِّل الذَّكِي

Ultra-Realistic Voice Generation and Text Annotation System for Quranic Translations

CSC 497 – Final Report

Prepared by:

Musaad Alquwayi	443101092
Ziyad Abdullah ALAbdullatif	443103765
Muhannad Saeed Alasmari	443102430
Yousef Saad Al-Dayhani	443101518
Faris Mohammed Alghofaili	442106146

Supervised by:

Dr. Mejdl Safran

Software project for the degree of Bachelor in Computer Science

Second Semester 1446/1447 Spring 2024/2025

Acknowledgements

We would like to express our heartfelt gratitude to our project supervisor, Dr. Mejdl Safran, not only for his invaluable guidance throughout the project but also for providing us with the opportunity to demonstrate our abilities. We would also like to extend our appreciation to King Saud University's Computer Science Department for their excellent teaching and the invaluable resources they have provided.

English Abstract

The voice translation for the Quran, especially for those who don't speak Arabic, is necessary to improve their understanding of the Quran and delivering the meaning of its words. So, in our project, we were aiming to build a web-based, ultra-realistic voice generation and text annotation system for Quranic translations, which would aim to provide solutions for multiple challenges, such as (1) Time intensive audio recording, which could be expensive both in time and money, requiring voice talent; (2) Changing nature of Quranic translation, where translators would have to update meanings due to the difficulties of translation to another language; and (3) Pronouncing Arabic words, due to how particularly challenging their pronunciations are (e.g. ,كهيعص ،الله) and that makes it harder for learners' simplification. Our solutions to these problems are (1) Tag-based text annotation; (2) A voice cloning engine; and (3) Contextual control. After we've offered several solutions for these problems with our proposed system, we experimented with three voice cloning models: StyleTTS2, F5-TTS, and Tortoise-TTS. With the help of Rukn El Hiwar Organization, we were able to train the models with a dataset of .wav files of surahs and verses of Quran voiced by Mikael Waters, with some of those files containing words with individually pronounced characters (e.g. ,كهيعص ،الله) to help with our tag annotation influence. After many experiments and training, and with the assistance of evaluation metrics, we chose StyleTTS2 as our optimal model due to its output speed. Preprocessing for datasets was also implemented to help with tag annotation. Finally, the final product is a website with a web-based system whose users can generate a voice file and annotate text to help with the translation of Quran for non-Arabic speaking Muslims.

Arabic Abstract

الترجمة الصوتية للقرآن الكريم ضرورية لتطوير فهم القرآن وتقديم معاني كلماته، خاصَّةً للأشخاص الذين لا يتحدثون العربية، ولذلك نحن في مشروعنا نهدف إلى بناء نظام على شبكة الانترنت لتوليد صوت واقعي لترجمات القرآن، تهدف هذه المنصة إلى تقديم حلولً لعدة تحديات، على سبيل المثال:

١- التسجيل الصوتي؛ لأنه من الممكن ان يكون مكلفاً للوقت والمال ويحتاج إلى موهبة صوتية.

٢- الطبيعة المتغيرة لترجمات القرآن؛ حيثُ أنَّ المترجمين عليهم تحديث المعاني بسبب صعوبات الترجمة من لغة لأخرى.

٣- طريقة نطق الكلمات العربية؛ بسبب دقَّةِ نطق بعضها مثل (الله، كهيعص)، وهذه الدقَّةِ تجعل تبسيطها أصعب للمتعلمين.

حلولنا لهذه المشاكل هي: أولاً، تعليق توضيحي نصي قائم على العلامات، ثانياً، محرك لاستنساخ الصوت، وأخيرًا، السيطرة السياقية. بعدما عرضنا عدَّة حلول لهذه المشاكل بالنظام الَّذي اقترحناه، جرَّبْنا ثلاثة نماذج لاستنساخ الصوت: (StyleTTS2)، (F5-TTS)، و(Tortoise-TTS)، بمساعدة جمعية ركن الحوار، استطعنا تدريب النماذج بمجموعةٍ من البيانات المكوَّنة من ملفات (wav.) تحتوي على سورٍ وآياتٍ قرآنية من أداء ميكايل واترز، وبعض هذه الملفات تحتوي على أحرفٍ مقطَّعة للمساعدة في التعليق التوضيحي النصِّي.

بعد عِدَّة تجارب وبعد التدريب، مع مساعدة مقاييس التقييم، اخترنا (StyleTTS2) لِيُمثِّل النموذج المُكَافِئ لنا، أيضًا تم تطبيق معالجة مسبقة للبيانات للمساعدة في التعليق التوضيحي النصِّي، وأخيرًا، المشروع النهائي هو موقعٌ بِنِظَام على شبكة الانترنت يستطيع مستَخدِميه توليدَ ملف صوتٍ ويُساعد في التعليق النصِّي التوضيحي للمساعدةِ في ترجَمَةِ القرآن للمسلمين الغير ناطقين للعربيَّة.

Table of Contents

Table of Figures

List of Tables

Chapter 1: Introduction

An accurate and phonetic Quranic translation, along with corresponding audio recordings, is difficult to find. The initial costs of translation and audio recording, as well as the ongoing need for updates, make this a time-intensive and costly task [1]. This makes it hard for individuals to learn from the Quran without an understanding of Arabic.

This project will introduce an ultra-realistic voice generation and text annotation system for Quranic translation. It is designed to overcome current hurdles in the process. The system will provide users with a wider range of control over speech, helping to reduce the costs associated with audio recording.

We will introduce several features that will add a significant level of customizability for the user. These features include the ability to choose different voices, add emphasis to speech using tags, and provide contextual control over the audio. Figure 2 shows the alternative paths. This approach will also help address the constant need to update audio recordings by reducing both the cost and time required for updates.

This project will provide an efficient and low-cost solution. This project will hopefully help people broaden their understanding of the Quran and build a foundation for similar projects in the future.

1.1 Problem Statement

The voice generator of Quran is necessary for those who does not speak Arabic very well because they need to know how Quran words sounds and like in their language and we try our best to make these requirement doable but we have difficulties like build a model that pronounce word right and that require much time recording and training out model on it which is also one of the difficulties.
Audio recording is a costly and time-consuming task, and this is especially true for translations from other languages. Although this doesn't change the fact that some text needs to be translated. This is especially true for the Quran. It's important for people to understand and learn from it. So, our project aims to create a web-based application for voice generation that will give people the ability to generate an audio file with their own customizations. The basic process is illustrated in figure 1.

The challenges that our web-based application will deal with:

- **Time Consuming Audio Recording:** Translating Quran by audio recording needs much time, high quality of recording equipment, recording environment, someone with a voice talent which means you need a lot of resources and budget.

- **Dynamic Nature of Quranic Translations:** The translator of Quran may update the words of Quran, so he changes it because the difficulty of translating it from a language to another so the problem here is to re-train the models to pronounce the unfamiliar word translated.

- **Lack of Control in Existing Solutions**: Modern voice cloning solutions don't provide the flexibility to customize voices. This is especially needed on context sensitive audio recording such as Quranic reading.

- **Challenges with Pronouncing Arabic Words:** Arabic words, particularly those found in the Holy Quran, are often challenging to pronounce due to the need for precise articulation. This makes it difficult to develop effective solutions that simplify the pronunciation of these words for learners and speakers.

1.2 Goals and Objectives

The goal is to develop a realistic voice synthesis system with a wide variety of voices and a text annotation feature for adding emotions, intonations, and other expressions. This involves designing an advanced voice synthesis system using deep learning, creating tools for dynamic voice control, and enabling contextual tagging to influence parameters like pitch, speed, and volume.

Goals:

- Realistic voice generating system with wide range of voices.
- Text Annotation system.
- Contextual control over voice parameters and using annotations to add emotions, intonations and other expressions.

Objectives:

- Design and implement a voice synthesis system capable of producing a wide range of realistic voices by integrating advanced deep learning algorithms.
- Create a text annotation feature that enables users to add context like emotions and intonations using tagging libraries.
- Develop a feature that allows users to dynamically control voice parameters such as pitch, speed, and volume, and utilize tags to influence these parameters.

1.3 Solution

To address the above challenges, we will provide a web-based system (basic design is shown on figure 3) that will try to resolve these challenges with mainly focusing on Quranic voice synthesizing. To resolve these challenges, we will provide these solutions:

- **Tag-based text annotation**: we will provide tags that will let the user choose from like "Pause" which stop reading like 1 second, "Allah" which magnify the word Allah, and these tags come from experts to add specific emotions, intonations,

expressions, and other prompts that will help users customize the audio at the word level.

- **Voice Cloning Engine:** we will build voice cloning engines that will have a prearranged set of voices. The user will be able to choose from these voices and a version of the Quran that will be used with it.

- **Contextual Control:** we will give the user the ability to edit the voice parameters. This will include:
    - Editing the tone.
    - Change the speed.
    - Adding pauses.
    - Emphasis on certain words.
    - And others.
- **Arabic Pronunciation Enhancement:** We will provide dedicated resources to ensure precise pronunciation of Arabic words, our platform also will include a comprehensive pronunciation guide and specific rules tailored to Quranic Vocabulary. A key challenge for learners is correctly articulating complex Arabic sounds, including the unique disjointed letters such as "الم" (Alif Lām Mīm) and "كهيعص" (Kāf Hā Yā ʿAyn Ṣād), which have no direct English counterparts. To address this, our platform leverages advanced machine learning models that are trained and retrained continuously to improve the accuracy and naturalness of Arabic pronunciation.

Proposed Solution Diagram:

Figure 1 shows the basic pipeline for using the web-based application. The user first logs in if s/he already has an account, or registers if s/he is a new user. Then, the user will choose a voice and a translation version of the Quran. The user then selects a surah, customizes the available parameters, and finally generates an audio file.

Figure 2 shows the alternative paths for the user (from left to right).

- The first shows the user selecting a version of the Quran and annotating it without generating audio.
- The second shows the user selecting a cloned voice and generating an audio file without any annotation.
- The third shows the user selecting a voice and version of the Quran and generating it after some annotations.

Figure 3 shows a theoretical representation of the web-based application.

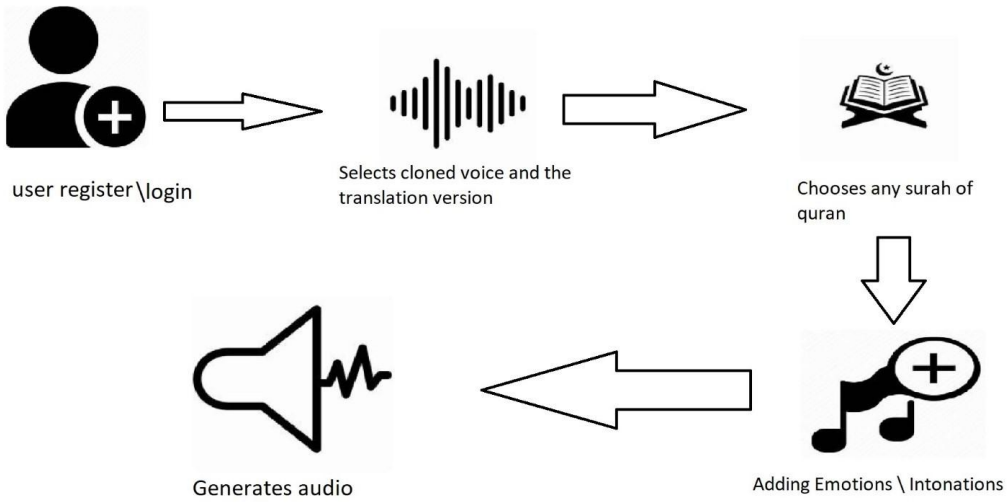

Figure 1: Proposed system diagram

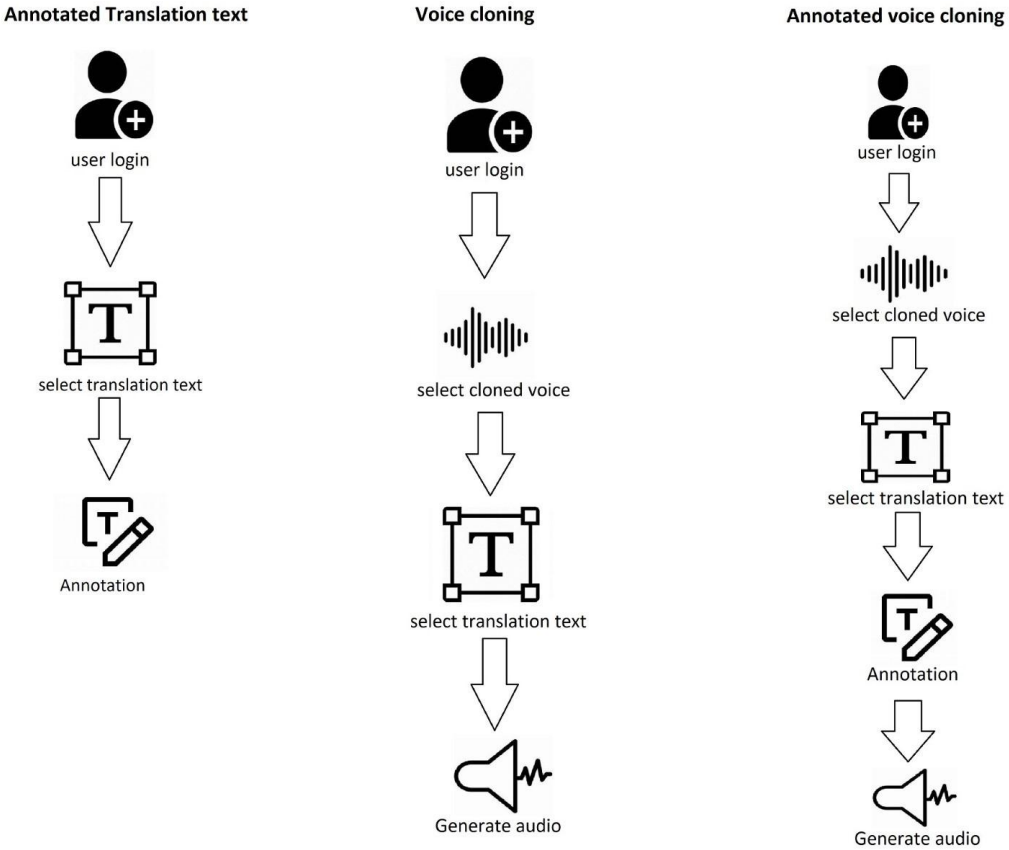

Figure 2: Proposed system diagram paths

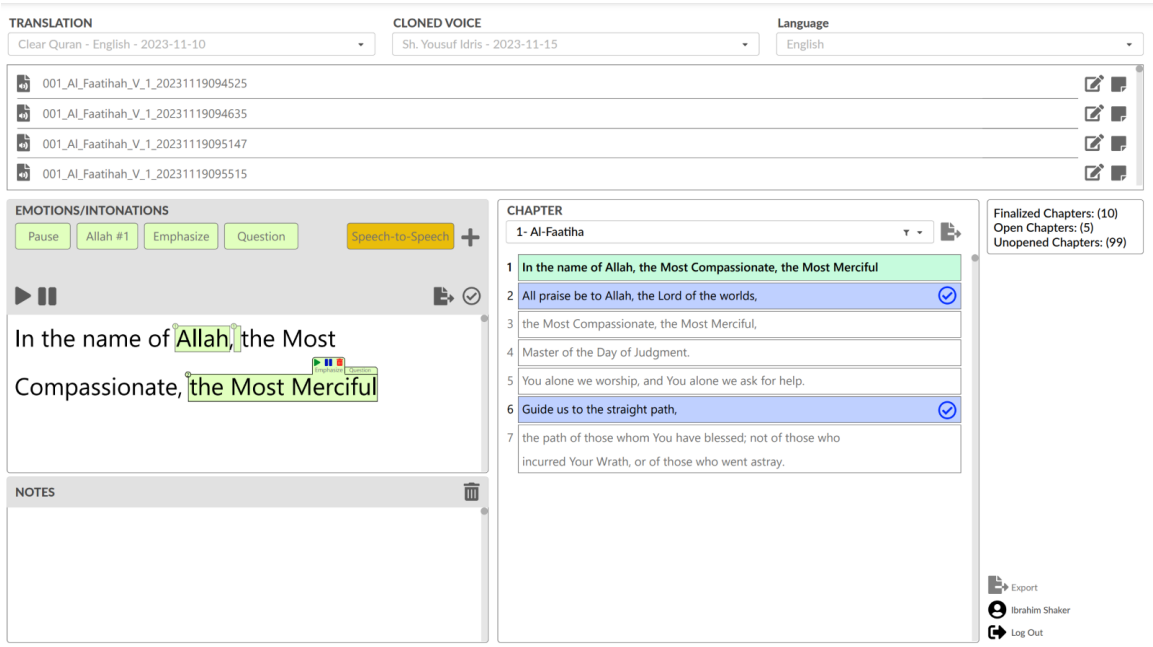

Figure 3: Imaginary image of main page [2]

1.4 Research Scope

Our project aims to create an Ultra-Realistic Voice Generation and Text Annotation System for Quranic Translations that will allow the user to choose a voice and pre generated tags and emotions.

The research limitations:

- Can only choose voice from our list of voices.
- Can only choose emotions from our list of emotions.
- Can add emotions only to certain words.
- Only available in certain languages.

Chapter 2: Background

The recording of Quranic verses after translations has not been to the expected par throughout history. Many issues get in the way of recording believable, accurately, and carefully pronounced Quranic verses and words, such as language and pronunciation barriers. But, since the rise of voice cloning and voice generation, ways to fix these daunting issues slowly started to emerge, accessible solutions for pronunciation difficulties, excessive costs of time and money, and swift fast changes have become readily available [3]. At the same time, text annotation assisted in that by providing transcriptions, characteristics, and information to replicate the voice more accurately in voice cloning, and to help the generated speech be more fluid and natural in voice generation [4].

So, the main idea here is to create a new accessible way to utilize this technology to deliver a web-based system used for Quranic translations, the necessity of which is due to the variety of both non-Arabic speaking and Arabic speaking Muslims who need a guidance to the proper way of reading Quran, the cost of finding and hiring talent for real-life voice recording sessions, and the everlasting nature of constant changes that happen to Quranic translations.

In this chapter we will provide a collection of definitions necessary to understand our research and project. Then we will discuss the Deep Learning concept and the layers necessary to make it work. We will also explain the concept of Voice Cloning, a breakdown of its innerworkings, and the three main architecture elements that make a voice cloning model work. Finally, we will discuss the concept of Text Annotation.

2.1 Definitions

- **Voice cloning:** Otherwise known as audio deepfake, voice cloning is the process of producing speech matching a target speaker voice, given textual input and an audio sample from the speaker [4]. And to capture the tone, accent, and the style of speaking.
- **Voice generation:** Voice generation is the artificial production of human speech [5]. However, this is a much broader term than voice cloning since it may or may not be modeled after a specific speaker. It could generate completely new voices.
- **Deep learning:** A subset of machine learning using neural networks with many layers to automatically extract complex patterns from data. It’s the part that makes computers use experience to their learning advantage.

- **Machine learning:** A broader field of AI which contains deep learning, where models learn from data to make predictions or decisions without explicit programming.
- **Dataset:** A collection of organized data that's typically used to train or evaluate machine learning models in various domains.
- **Text annotation:** The process of labeling text with specific tags or information to train models for natural language processing tasks. It helps in voice cloning by providing accurate transcription of words being spoken and capturing some of their unique ways, and in voice generation by including data on accents and dialects and generating speech that sounds natural.
- **Training data sets:** The labeled data is used to teach machine learning models to recognize patterns and make predictions during the learning process.

2.2 Deep learning

Deep Learning (DL) is a subset of Machine Learning (ML), and both are types of AI. Basically, it enables computers to learn from experience. One of its components is Neural Network (NN) which is a way to simulate the brain of humans. It's made up of very huge number of linked processing nodes called neurons. They process data in three or more layers [6], and the layers are:

Input Layer: Receive input and nodes process the data, analyze, or categorize it, and pass it on to the next layer.

Hidden Layer: Receive input from input layer or other hidden layers and process it and pass it to next layer or output layer.

Output layer: The output layer will give result of all data processing by AI and number of nodes depend on problem classification like if we have binary (yes/no) classification problem, the output layer will have one output node, which will give the result as 1 or 0 [6].

As shown in Figure 4 we see two neurons in input layer take input and Prosses it and pass it to Hidden layer and Prosses it and pass it to output layer which is the result.

So Neural Network is good for recognizing patterns and plays an important role in applications including natural language translation, image recognition, speech recognition, and image creation and it's used in ML and DL [7].

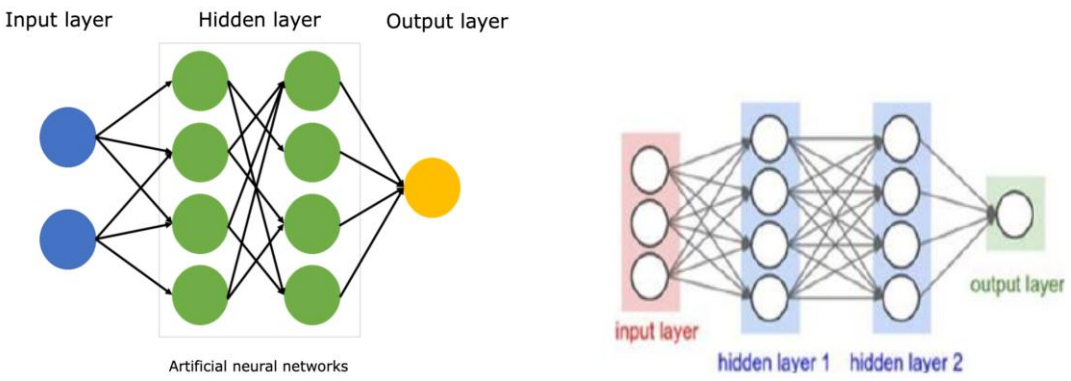

Figure 4: Images of Neural networks and Deep learning [8] [9].

2.3 Voice Cloning

Voice Cloning is the task of learning to synthesize the voice of speaker from a sample of his voice [10]. Voice cloning is a complex process here is breakdown of how it's work:

1. **Voice Sampling:** The first step is collecting a dataset which is amount of audio from the person whose voice is to be cloned and transcribed of voices [11].
2. **Audio Analysis:** The collected voice is analyzed and this analysis "involves breaking down the audio into phonemes (the smallest units of sound in a language) and understanding various characteristics like pitch, tone, and speed [11].
3. **Feature Extraction:** After the analysis, the distinctive features of the voice like Mel_Spectogram are extracted. These features include unique aspects like accent, intonation, and rhythm, which make each voice recognizable [11].
4. **Training the AI Model:** The extracted features are used to train an AI model, typically a type of neural network. This training process involves the model learning to replicate the specific characteristics of the voice [11].
5. **Synthesis and Fine-Tuning:** Once the AI model is trained, it can generate new speech in the cloned voice. This speech is then fine-tuned to ensure it sounds natural and matches the original voice's nuances [11].
6. **Output Generation:** The final step is the AI model producing the cloned voice output [11].

There are three main architecture elements in every voice cloning model which are:

Encoder: First speaker encoder its goal is to create in a latent, fixed-dimensional embedding (vector) that captures various features of a particular human voice [12].

Synthesizer: Its goal is to synthesize a Mel-Spectogram from a text transcript [12].

Vocoder: Its goal is to make an audio waveform from the previously synthesized Mel-Spectrogram [12].

As shown in Figure 5 we see architecture represented as a block diagram. The green, blue and red blocks denote the Encoder, Synthesizer and Vocoder units. This model is called SV2TTS.

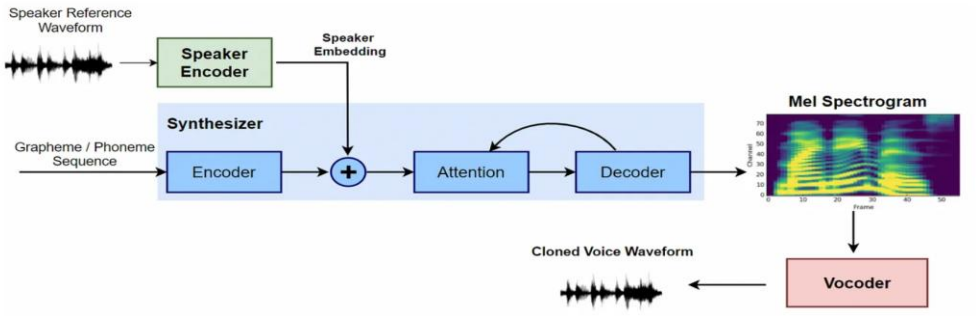

Figure 5:Image of how voice cloning works [13]

2.4 Text Annotation:

Text annotation: is the process of assigning labels to a text or different elements of its content to identify the characteristics of sentences [14]. One major type in text annotation is called **Entity Notation (**EN) which is the process of assigning entities in text with their corresponding predefined labels based on their semantic meaning [14] as shown in Figure 6 as we can see the tagged words and we can put tags.

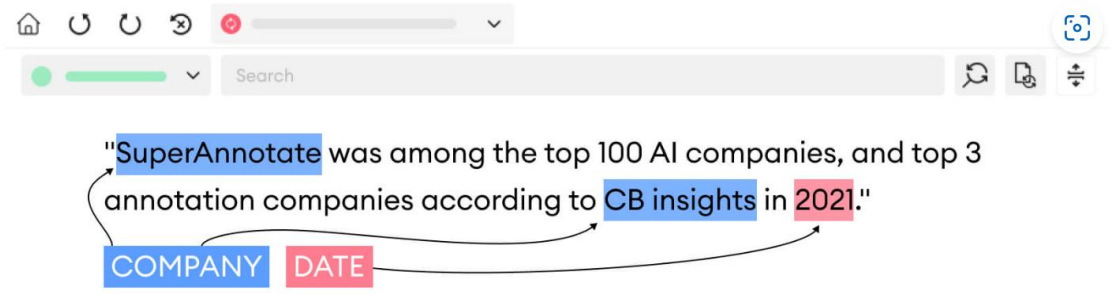

Figure 6: Imaginary image Entity Notation [14]

Chapter 3: Literature Review

In this chapter we will review and discuss related research articles and software programs in the scope of Voice cloning and Text Annotation.

3.1 Related Work (Research)

3.1.1 Voice Cloning

In [15] the authors discuss the challenge of generating natural, high-quality speech directly from text in text-to-speech (TTS) systems. Tacotron2 is what they proposed for this problem. Figure 7 Shows the architecture of Tactorn2, combines a recurrent sequence-to-sequence network with attention to predict mel spectrograms from text, followed by a modified WaveNet vocoder to generate time-domain waveforms from the spectrograms. This fully neural approach simplifies the traditional TTS pipeline by eliminating the need for complex linguistic and acoustic features, allowing the system to be trained directly from data. The result shows Tacotron2 achieves a mean opinion score of 4.53, that is close to 4.58 for professionally recorded speech.

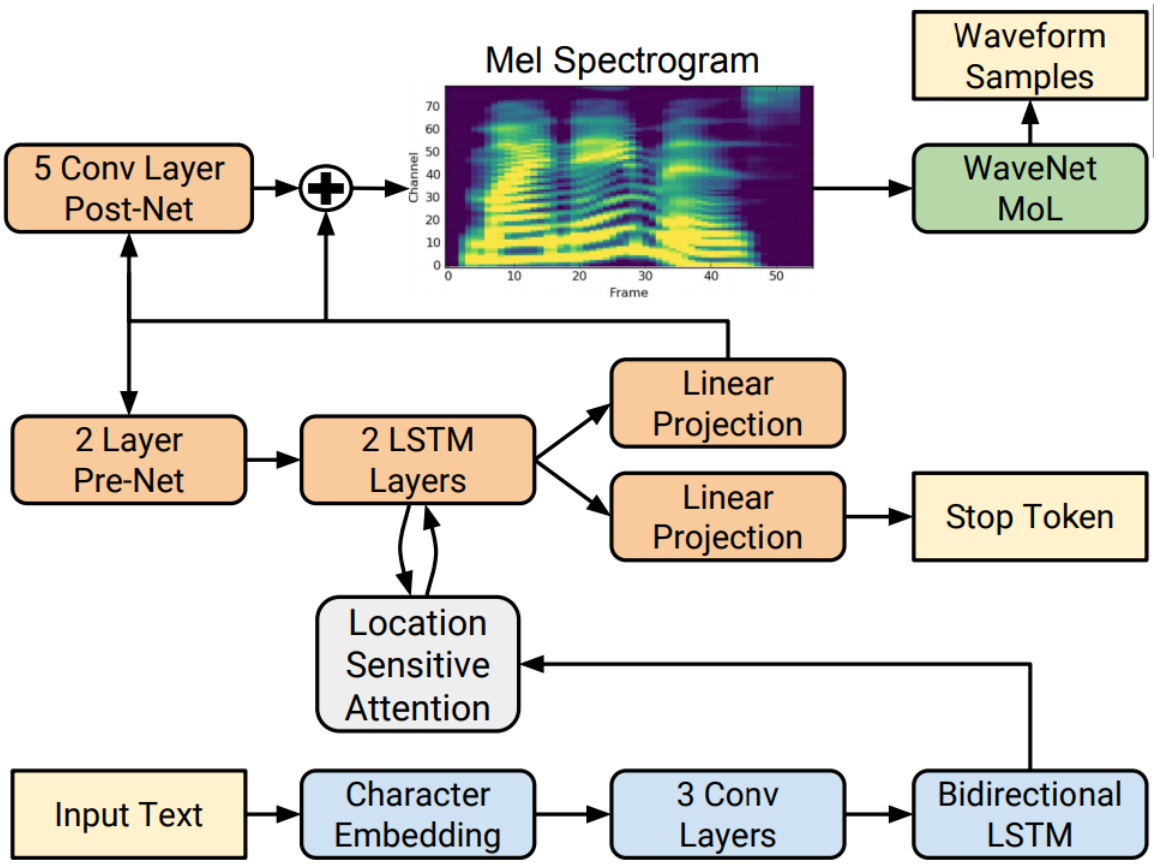

Figure 7:Tactorn2 architecture [15]

The authors in [16] introduced text to speech model (TTS) based on Tacotron2 and Transformer architectures. Figure 8 shows the architecture of Transformer. The model addresses two challenges (1) low training efficiency (2) hard to model long dependency using current recurrent neural networks (RNNs). Figure 9 shows the architecture of the model. It replaces the RNN structures and the original attention mechanism in Tacotron2 with a multi-head attention mechanism, enabling parallel training and improving the ability to capture global context in audio synthesis. Any two inputs at different times are connected directly by a self-attention mechanism, which solves the long-range dependency problem effectively. By Using phoneme sequences as input, the Transformer TTS network generates mel spectrograms, followed by a WaveNet vocoder to output the final audio results. For efficiency the Transformer TTS network speed up the training about 4.25 times faster compared with Tacotron2. The tests show that model achieves state-of-the-art performance outperforms Tacotron2 with gap of 0.048 and very close to human quality (4.93 vs 4.44 in MOS).

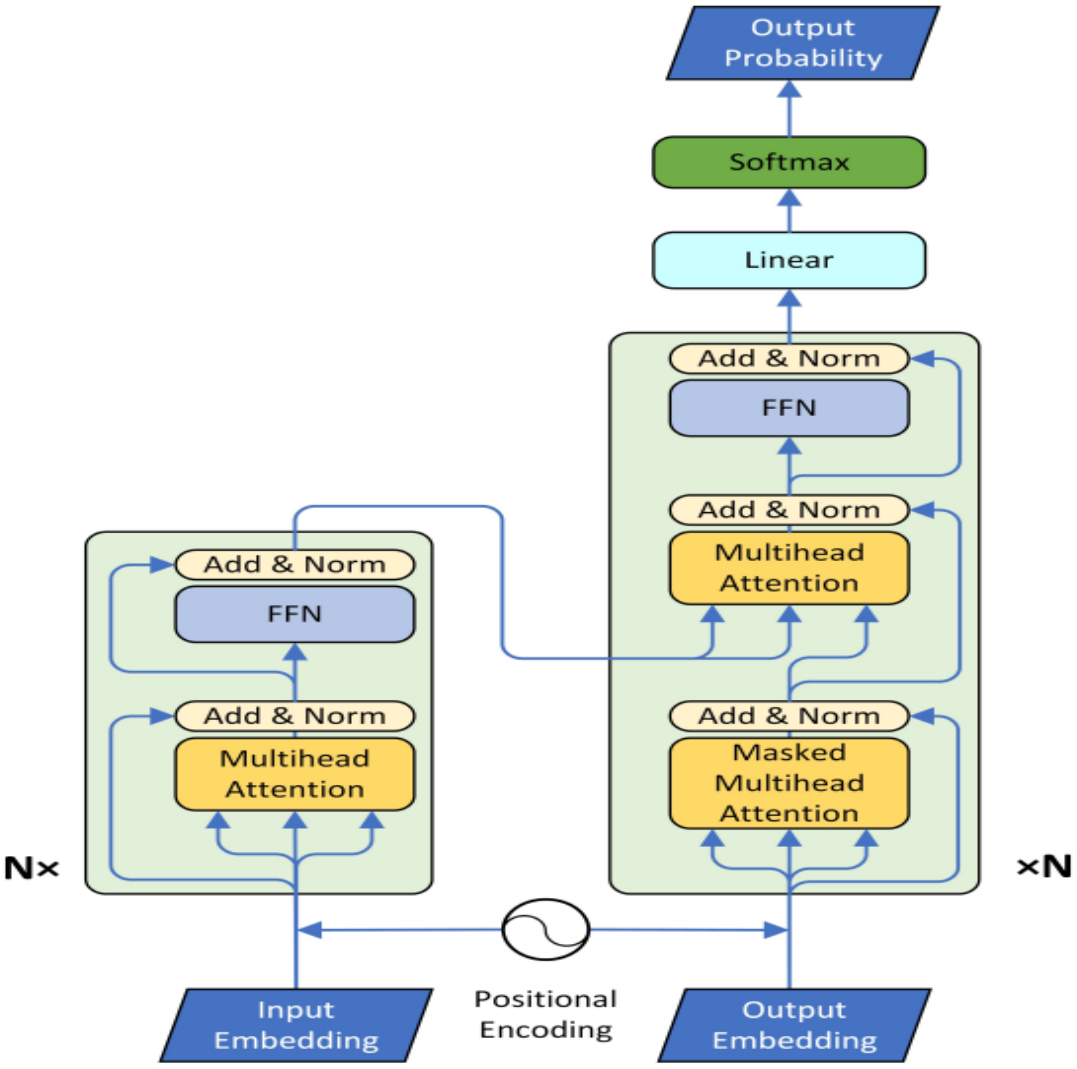

Figure 8: System architecture of Transformer [16]

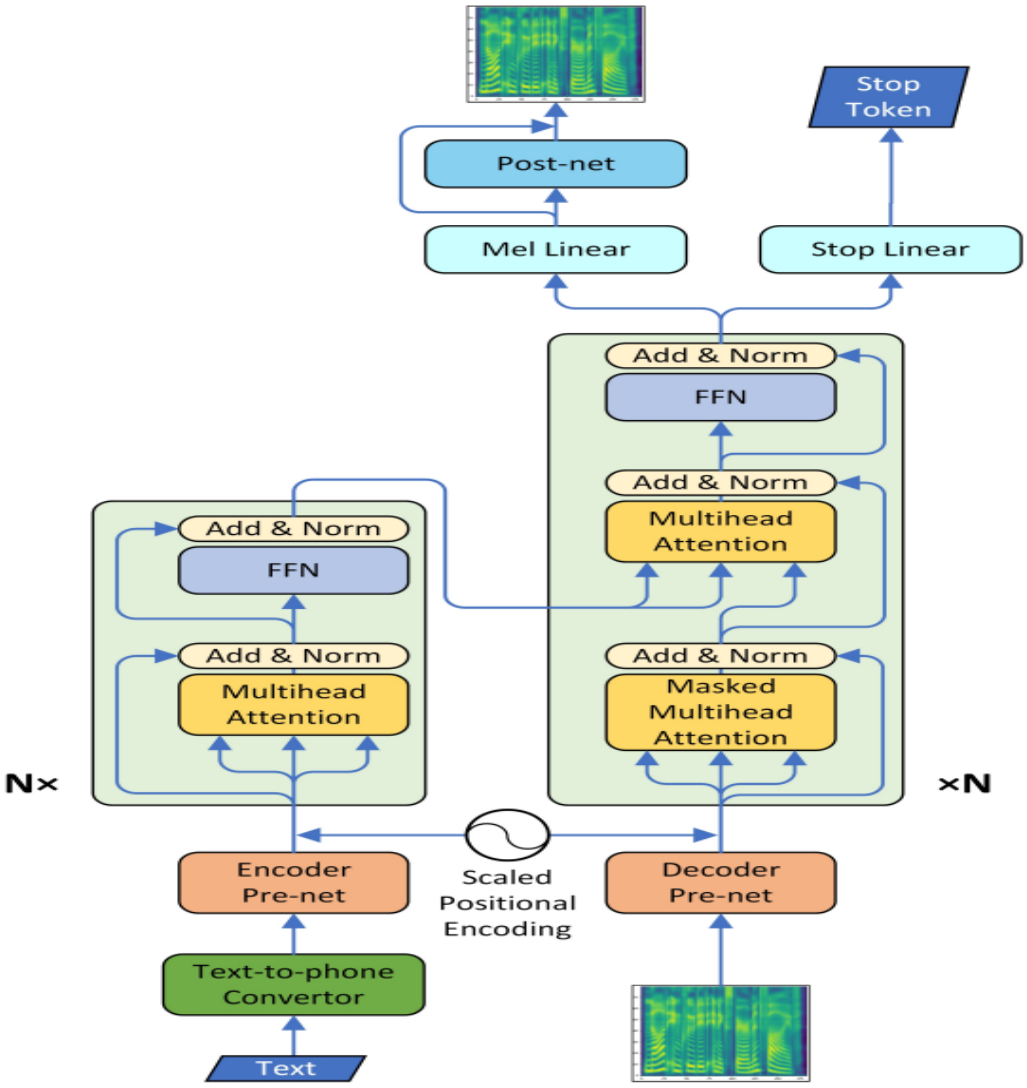

Figure 9: System architecture of the model [16]

The authors in [17] introduced two approaches for voice cloning (1) Speaker Adaptation and (2) Speaker Encoding. Summarized of two approaches in figure 10. In Speaker Adaptation, a pre-trained multi-speaker model is fine-tuned to match the voice characteristics of a new speaker using a small set of audio-text pairs. This can involve adjusting either the speaker embedding or the entire model. Fine-tuning the embedding alone updates the speaker's unique voice features while keeping the rest of the model intact, allowing for high-quality voice cloning with minimal data. In the Speaker Encoding method, which directly estimates, scratch. Table 1 shows the comparison between these approaches and lists the requirements for training, data, cloning time and memory footprint.

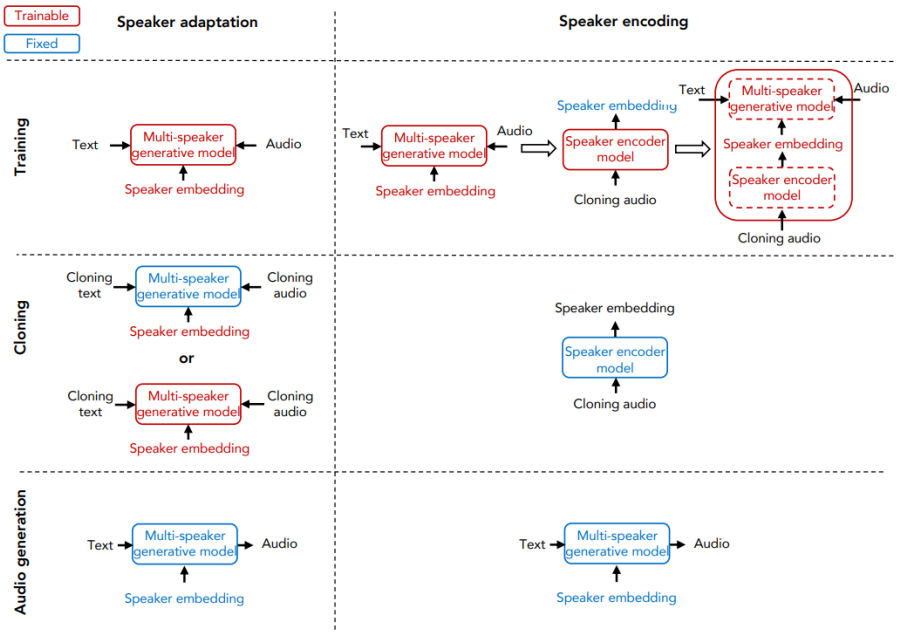

Figure 10: Illustration of speaker adaptation and speaker encoding approaches for voice cloning [17]

Table 1: Comparison of speaker adaptation and speaker encoding approaches [17]

	Speaker adaptation		**Speaker encoding**	
Approaches	Embedding-only	Whole-model	Without fine-tuning	With fine-tuning
Data	Text and Audio		Audio	
Cloning Time	~ 8h	~ 0.5 − 5m	~ 1.5 − 3.5s	~ 1.5 − 3.5s
Inference time	~ 0.4 − 0.6s			
Parameters per speaker	128	~ 25 million	512	512

The authors in [18] introduced three main tasks to evaluate voice cloning methods:

1- **Text-Based Cloning**: Speech synthesis is achieved from textual input through a text-to-speech (TTS) model, such as Tacotron 2 combined with WaveGlow, which has been trained on datasets like LJ Speech. The generated speech's pitch is modified to align with that of the intended speaker, while rhythm is synchronized using a forced alignment model. To maintain the distinctive style of the speaker, a Global Style Token (GST) embedding is utilized, derived from samples of the target speaker.

2- **Imitation Setup**: A text-audio pair from the target speaker, which was not included in the training dataset, is synthesized using the established model. The pitch, rhythm, and GST embedding are extracted from the target speech to assess the degree to which the synthesized output replicates the speaker's stylistic characteristics.

3- **Style Transfer Task**: The pitch and rhythm characteristics from an expressive speaker, such as those found in the Blizzard 2013 dataset, are applied to the cloned speech of the target speaker. The reference audio for style extraction aids in obtaining these features and adjusting them to correspond with the target speaker's average pitch, while the GST embedding ensures that the unique stylistic elements of the target speaker are preserved.

The authors in [19] introduced a new approach for Voice Cloning (VC) by addressing scenarios that cannot be handled well by traditional VC methods, such as movie dubbing, where speech needs to convey emotions aligned with the plot. Existing Voice Cloning tasks aim to convert a paragraph of text into speech using a desired voice specified by a reference audio, significantly boosting the development of artificial speech applications. However, they fall short in scenarios requiring consistent emotional expression.

To fill this gap, the authors proposed a novel task called Visual Voice Cloning (V2C), which seeks to convert a paragraph of text into speech with both the desired voice (specified by reference audio) and the desired emotion (specified by reference video). This new approach introduces a unique way of integrating multimodal inputs for more emotionally expressive voice synthesis.

As presented in Table 2, various datasets are analyzed in terms of their support for multimodal input (e.g., text, audio, video) and key properties such as the number of speakers and clips. Among these, the V2C-Animation dataset stands out with its combination of audio, video, and emotion labels, specifically designed to support the V2C task. The dataset includes 10,217 video clips that cover diverse genres and emotions, providing a foundation for evaluating emotionally expressive voice cloning.

To support this research, a strong baseline model based on state-of-the-art VC techniques was proposed to establish performance benchmarks. To evaluate the effectiveness of the generated speech, the authors designed a set of evaluation metrics named MCD-DTW-SL to measure the similarity between ground-truth speeches and synthesized ones. Extensive

experimental results show that even state-of-the-art VC methods struggle to produce satisfactory speeches for the V2C task.

The authors hope that the proposed V2C task, along with the V2C-Animation dataset and the evaluation metrics, will advance research in voice cloning and benefit the broader vision-and-language community.

Table 2: The author compares V2C-Animation dataset with several existing multi-modal datasets [19]

Dataset	Text	Audio	Video	Identity	Emotion	#Movies	#Video Clips	#Audio Clips	#Speakers	Avg. S	Avg. A/V (s)
LJ Speech [19]	√	√		√		-	-	13100	1	17.23	6.57
LibriSpeech [31]	√	√		√		-	-	250698	2484	32.55	14.10
VCTK [49]	√	√		√		-	-	44070	108	7.41	3.59
LibriTTS [51]	√	√		√		-	-	375086	2456	16.86	5.62
MPII-MD [36]	√		√			94	68337	-	-	-	3.88
MovieQA [41]	√		√			140	6771	-	-	6.20	202.67
LRS2-main [10]	√	√	√			-	48164	48164	-	7.13	1∼2
LRS2-pretrain [10]	√	√	√			-	96318	96318	-	21.43	∼10
V2C-Animation	√	√	√	√	√	26	10217	10217	153	6.51	2.40

The authors in [20] introduced a novel approach for text-to-speech (TTS) synthesis called FastSpeech, addressing three main challenges in neural network-based TTS systems: (1) slow inference speed, (2) lack of robustness, and (3) limited controllability. FastSpeech achieves faster inference by generating mel-spectrograms in parallel, compared to traditional autoregressive models, which are often slowed by their sequential nature. As illustrated in Figure 11, FastSpeech utilizes a feed-forward Transformer structure that incorporates a phoneme duration predictor and a length regulator to match the input phoneme sequence with the output mel-spectrogram length. This setup enables more control over speech aspects, such as speed and prosody, by adjusting phoneme duration directly. Table 3 compares FastSpeech's audio quality, showing it performs nearly as well as autoregressive models while significantly reducing latency and minimizing skipped or repeated words.

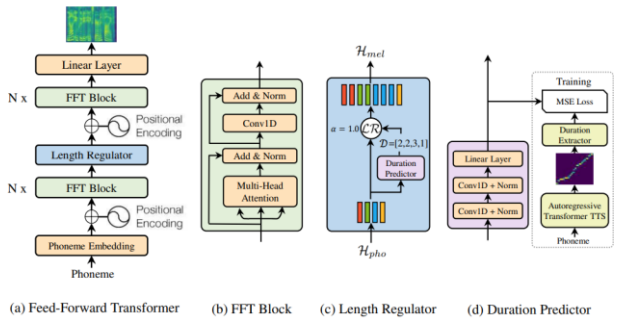

Figure 11: The overall architecture for FastSpeech [20].

Table 3: The MOS with 95% confidence intervals [20].

Method	MOS
GT	4.41 ± 0.08
GT (Mel + WaveGlow)	4.00 ± 0.09
Tacotron 2 [22] (Mel + WaveGlow)	3.86 ± 0.09
Merlin [28] (WORLD)	2.40 ± 0.13
Transformer TTS [14] (Mel + WaveGlow)	3.88 ± 0.09
FastSpeech (Mel + WaveGlow)	3.84 ± 0.08

The authors in [21] introduced WaveGlow a flow-based network capable of generating high quality speech from mel spectrograms.WaveGlow combines insights from Glow and WaveNet to provide fast, efficient and high-quality audio synthesis, without the need for auto-regression. WaveGlow is implemented using only a single network, trained using only a single cost function which maximizes the likelihood of the training data, which makes the training procedure simple and stable. The model uses an invertible convolution and affine coupling layers to transform Gaussian noise into realistic audio, as shown in Figure 12. This approach allows WaveGlow to deliver superior performance with minimal computational cost, achieving a synthesis rate of over 500 kHz on an NVIDIA V100 GPU, as detailed in Table 4. Compared to autoregressive models, WaveGlow simplifies training while maintaining high audio quality.

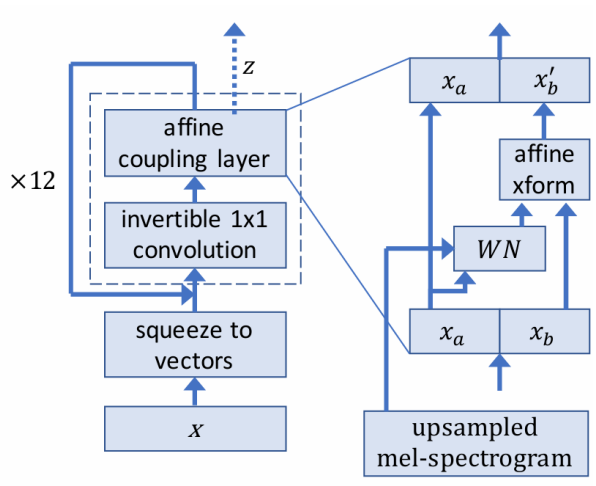

Figure 12: WaveGlow network [21].

Table 4: WaveGlow Mean Opinion Score (MOS) [21].

Model	Mean Opinion Score (MOS)
Griffin-Lim	3.823 ± 0.1349
WaveNet	3.885 ± 0.1238
WaveGlow	3.961 ± 0.1343
Ground Truth	4.274 ± 0.1340

3.1.2 Text Annotation

The authors in [22] introduced DoTAT, a domain-oriented text annotation tool designed to address the specific needs of domain-related information extraction tasks. The tool implements several critical functions for structuring domain-specific texts effectively.

As shown in Figure 13, DoTAT facilitates a collaborative annotation process involving multiple annotators and a reviewer. The annotation process begins with the administrator defining annotation specifications for entities, relations, events, and classifications. Tasks are then created and assigned to annotators, who proceed to annotate raw text following the defined specifications.

The tool supports complex annotations such as nested entities and events, enhancing its applicability to domain-oriented projects. After annotation, a reviewer checks for consistency and merges annotations where necessary, ensuring high-quality results. Finally, the results can be exported, and the workflow can iterate as required, integrating feedback for improved accuracy.

Additionally, DoTAT introduces several efficiency-boosting features like automatic batch annotation, iterative annotation, and visual specification definitions, which contribute to its streamlined workflow.

Experiments conducted on the ACE2005 dataset reveal that DoTAT reduces annotation time by 19.7% compared to existing tools. It achieves an accuracy of 84.09% without review, surpassing Brat and Webanno. When the review process is included, DoTAT's accuracy rises to 93.76%, establishing it as an efficient and highly accurate solution for domain-specific text annotation.

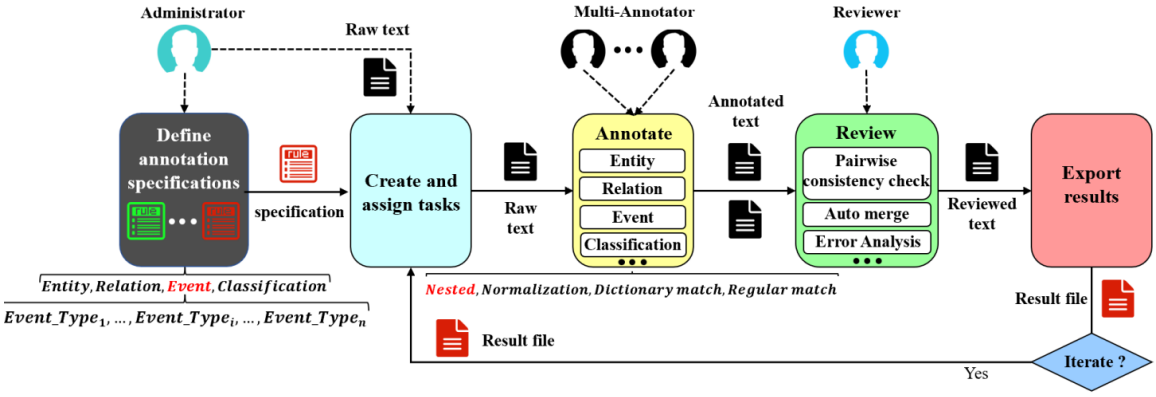

Figure 13: Typical workflow using DoTAT [22]

The authors in [23] introduced TeamTat, a new web-based annotation tool designed to streamline the development of text-mining and information-extraction algorithms by facilitating efficient multi-user annotations. Manually annotated data is essential for building these algorithms, but human annotation requires substantial time, effort, and expertise. With the rapid growth of biomedical literature, existing text annotation tools often fall short in supporting collaborative workflows and managing large-scale projects effectively.

As depicted in Figure 14, TeamTat facilitates the entire annotation lifecycle through a comprehensive project management system. Project managers can set up a project, upload documents, and distribute them to multiple annotators, ensuring efficient task management. The tool supports the annotation of relations, overlapping annotations, and concept annotations at various levels, such as sentence, paragraph, or document level.

During the annotation rounds, annotators work independently in their personalized workspaces to process documents. At the end of each round, the project manager collects the annotations, reviews the project's status, and assesses progress using built-in quality control features like inter-annotator agreement statistics. Additionally, the tool supports multiple input formats and automatically retrieves documents from PubMed/PMC, making it convenient for biomedical literature projects.

TeamTat allows for seamless collaboration, even for large projects, while maintaining high-quality annotations through its robust management and quality assurance features. This makes it a valuable platform to support the growing demands of biomedical literature annotation and large-scale text-mining tasks.

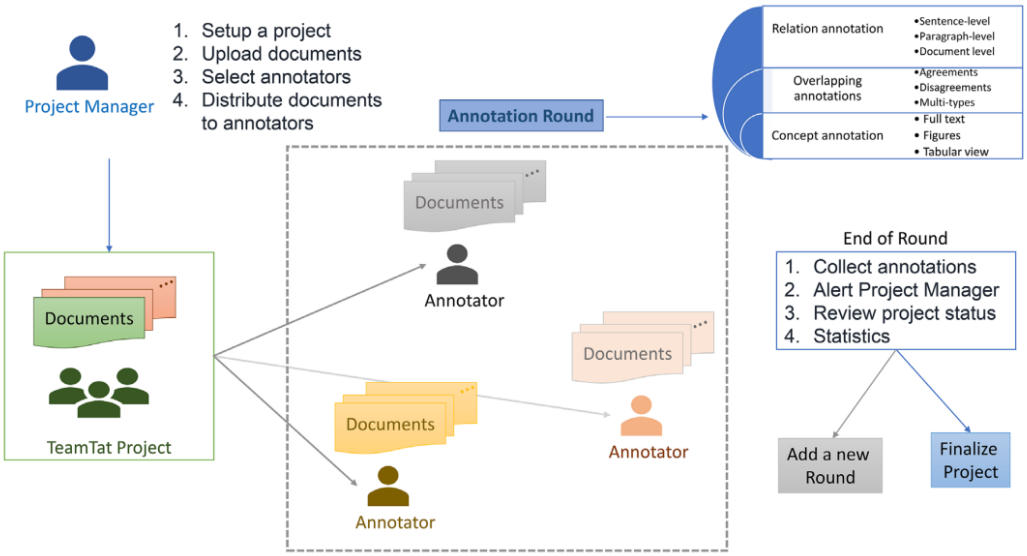

Figure 14: Overview of a TeamTat annotation project [23]

The authors in [24] introduced FITAnnotator, a generic web-based tool for efficient text annotation. FITAnnotator provides efficient solutions for the annotation of a variety of natural language processing tasks, including classification, sequence tagging and semantic role annotation, regardless of the language. The system integrates active learning to minimize manual labeling efforts by selecting and prioritizing the most informative samples, thus reducing the total number of annotations required for training machine learning models. It also incorporates incremental learning, which allows for the continuous integration of new classes without needing to re-annotate existing data, making it suitable for dynamic real-world applications. FITAnnotator is designed for multi-user collaboration, providing distinct interfaces for annotators, reviewers, and managers to coordinate tasks effectively. Furthermore, the system's adaptability to various languages and tasks is achieved through its fully modular architecture, enabling seamless customization and expansion. Figure 15 illustrates the interaction between active learning, incremental learning, and crowdsourcing, which together form the core of FITAnnotator's intelligent annotation system. Figure 16 shows the overall architecture of the system.

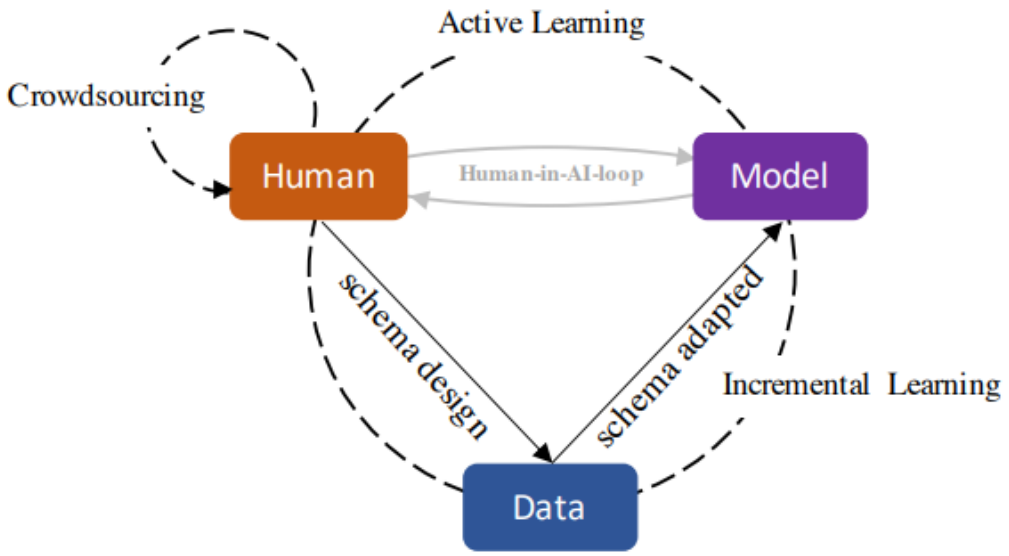

Figure 15: The interaction of the three major elements of the intelligent annotation system [24]

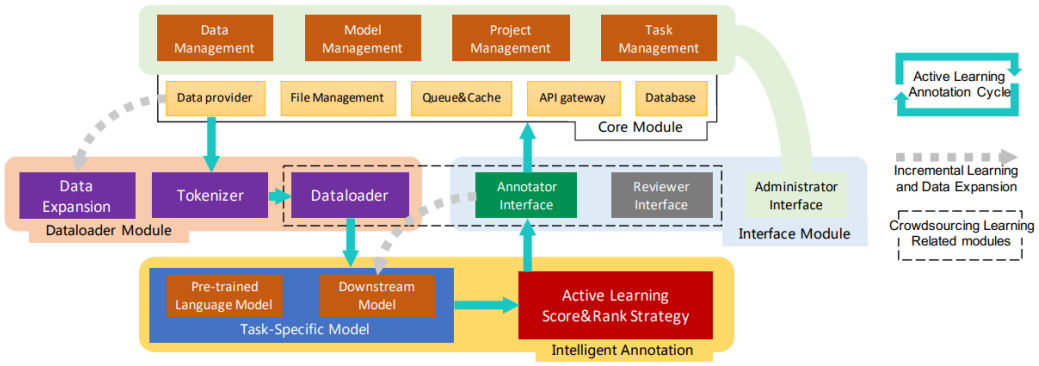

Figure 16:The overall architecture of the system [24]

3.2 Related Work (Software)

3.2.1 Voice cloning

- **Resemble AI** [25] is an advanced platform specializing in voice cloning and text-to-speech (TTS) technologies, aimed at generating high-quality synthetic voices with minimal data requirements. Its notable feature is the capability to replicate a voice using just a few minutes of recorded audio, which enhances its accessibility and efficiency for a wide range of applications. Figure 17 shows the main screen.

Features:

- One of the notable aspects of Resemble AI is its capability to produce voices that convey various emotional tones. Users have the option to tailor both the style and emotional expression of the replicated voice figure 18 shows that.

- The platform facilitates the cloning of voices in various languages, allowing users to generate voice clones that are multilingual.

- Ease to use.

Drawbacks:

- Resemble AI offers a free version; however, it imposes limitations on certain features. To gain access to advanced functionalities and increased usage limits, users are required to subscribe to a paid plan.

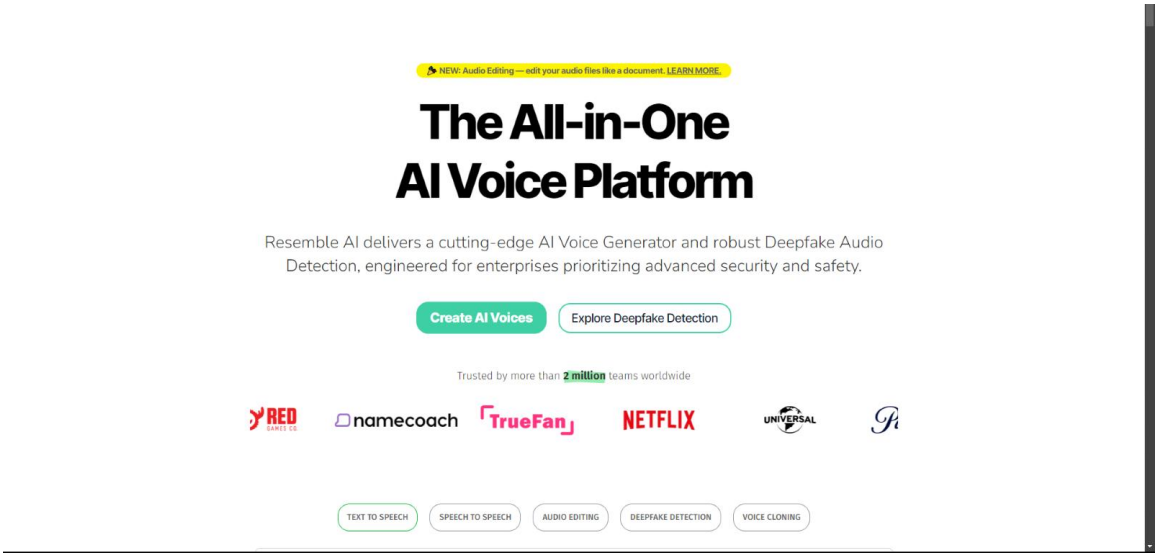

Figure 17:Resemble AI [25]

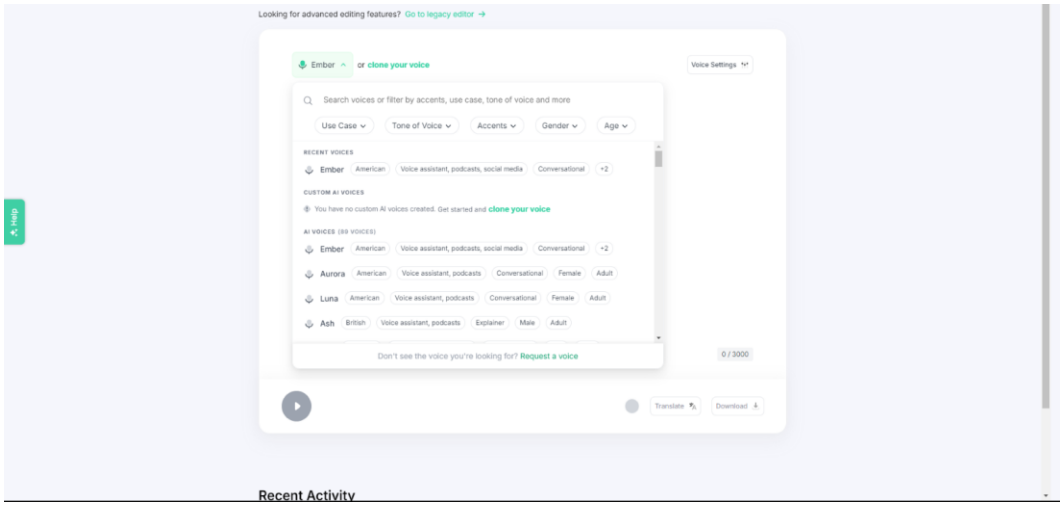

Figure 18: Resemble AI [25]

- **Speechify** [26] offers a user-friendly voice cloning service that utilizes AI to replicate a user's voice, making it possible to generate new audio content quickly and easily. This technology allows users to upload a short audio sample or record directly through a microphone. After analyzing this data, the system can produce speech that mimics the original voice, maintaining its unique tone and style. As demonstrated in Figure 19, the process begins with importing the user's voice, either by uploading a file or recording directly. The system then proceeds to generate audio with the cloned voice, allowing users to input text and listen to the results. This streamlined interface makes it simple for content creators and professionals to edit or generate voiceovers without the need for additional recording sessions. With commercial usage rights and scalability options, Speechify is especially beneficial for last-minute voiceover adjustments. As seen in Figure 20, Speechify AI voice cloning service provides access to features such as commercial usage rights, the ability to maintain voice nuances, and fast audio generation, making it ideal for content creators and professionals needing scalable voice solutions.

  Features:

  - **Text-to-Speech Conversion:**
  -
    - Speechify can read aloud various types of text, including documents, articles, PDFs, and web pages. This helps users consume content hands-free, whether for productivity or accessibility purposes.

- **Wide Range of Voices:**

    - It offers a variety of high-quality, natural-sounding voices, including celebrity voices like Gwyneth Paltrow and Snoop Dogg. Premium users have access to 130+ voices in over 30 languages.

Drawbacks:

- **Limited Free Version:**

    - While Speechify offers a free version, access to the most advanced features, including premium voices, requires a subscription.

- **Limited Domain-Specific Features:**

    - The tool is not specialized for domain-specific tasks such as medical, legal, or technical document processing, which might limit its effectiveness in professional or academic use cases.

- **Pricing:**

    - **Free Version:** Users can access basic text-to-speech conversion features with a limited selection of voices for free.
    - **Premium Subscription:** A paid subscription unlocks access to over 130 voices, including celebrity voices, and support for multiple languages, with additional features like faster processing and higher-quality audio

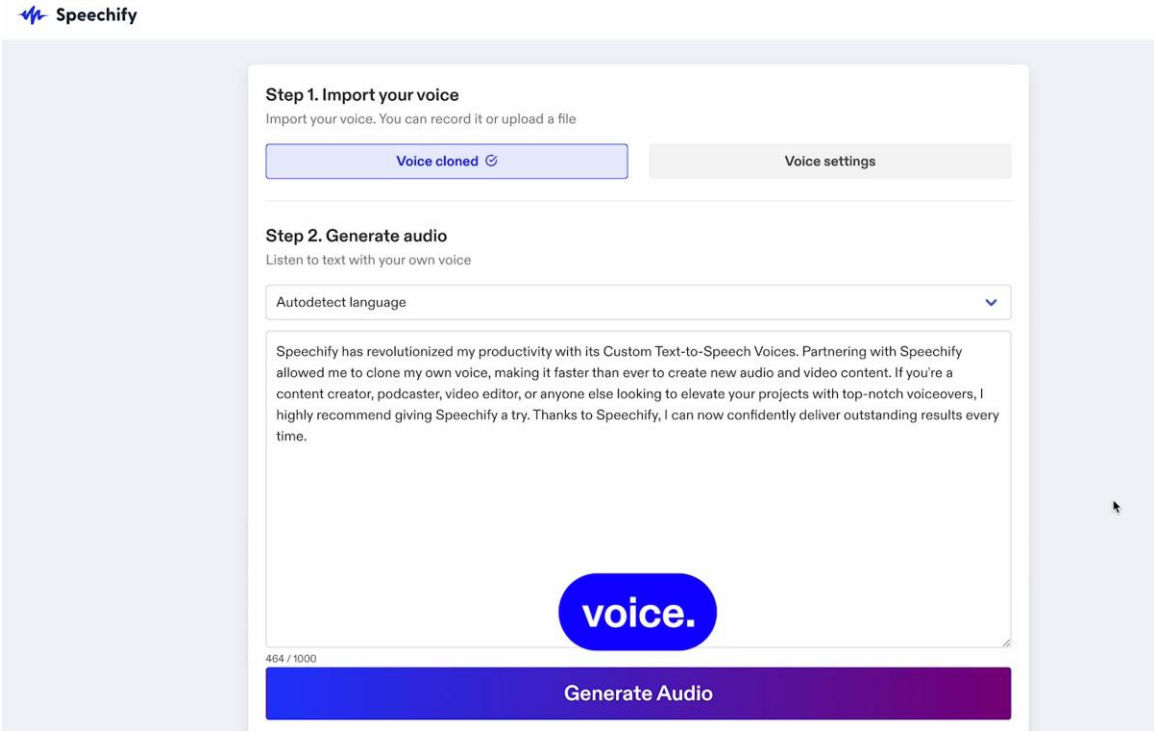

Figure 19: Speechify [26]

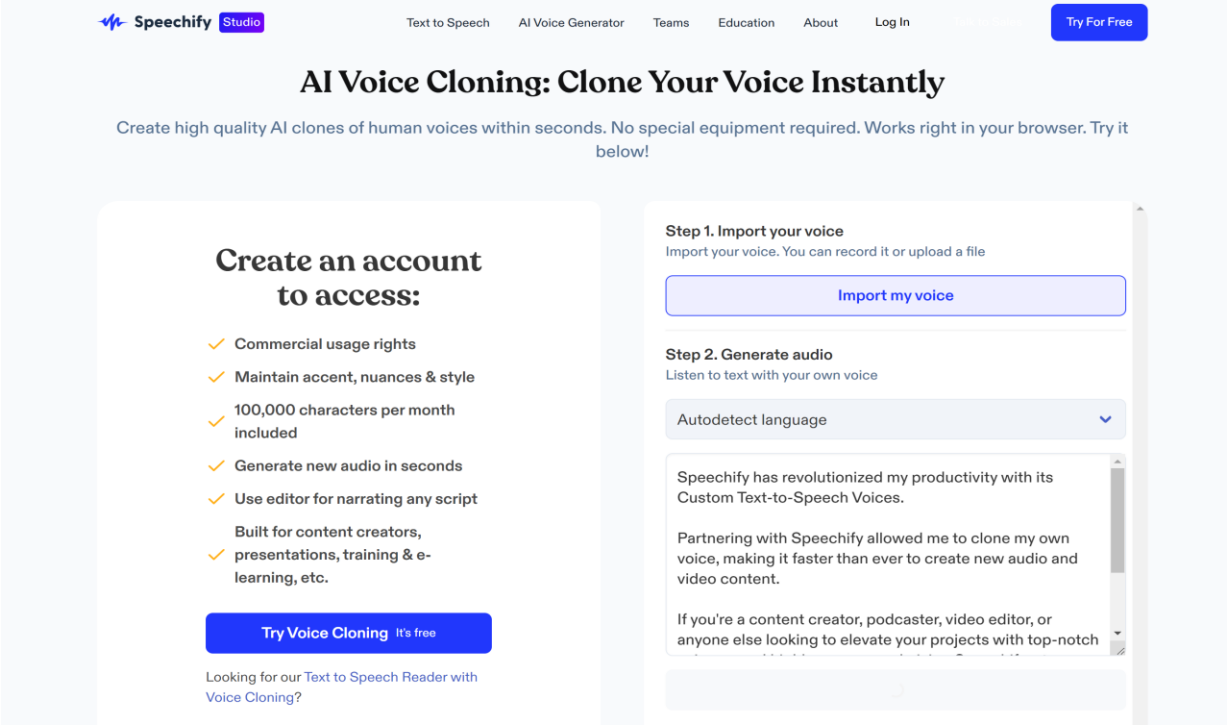

Figure 20: Speechify [26]

- **Descript** [27] offers a robust voice cloning tool called Overdub that allows users to create high-quality AI clones of their voices or use a library of stock voices. With Overdub, users can easily replace or correct audio mistakes by typing the desired text, making it an invaluable tool for podcasters, video editors, and content creators. As demonstrated in Figure 21, Overdub integrates seamlessly with Descript's text-based editor, where users can select voices, type in corrections, and instantly generate new audio that aligns with the video or project, simplifying the editing process.

Features:

- **Overdub for Voice Editing:**
    - You can fix verbal errors, change parts of your script, and even add new sections using your cloned voice without re-recording. It's designed to save time and maintain consistency in the audio.

- **Custom Voices:**
    - With Descript, you can generate realistic AI voices that vary in tone, style, and pacing. This is perfect for creating voiceovers for different types of media content, from corporate presentations to video narration.

Drawbacks:

- **Limited Free Version:**
    - While Descript offers a free version, access to advanced features, like custom voice cloning and higher voice quality, is limited to paid plans.

- **Accuracy Limitations:**
    - In certain cases, especially with more complex scripts or specific pronunciations, Overdub might not perfectly replicate the nuances of a natural voice, requiring further manual adjustments.

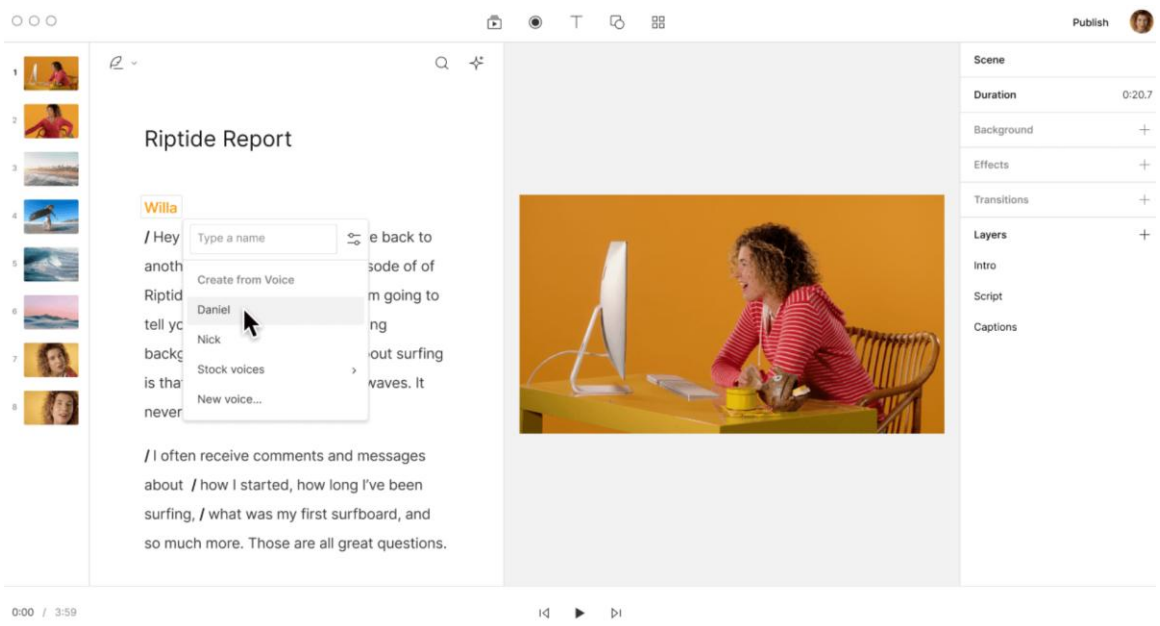

Figure 21: Descript [27]

- **TensorFlowTTS** [28] TensorFlowTTS provides real-time state-of-the-art speech synthesis architectures such as Tacotron-2, Melgan, Multiband-Melgan, FastSpeech, FastSpeech2 based-on TensorFlow 2.

  Features:
  - High performance on speech synthesis.
  - Fast, scalable, and reliable.
  - Suitable for deployment.
  - Easy to implement a new model, based-on abstract class.

  Drawbacks:
  - Support limited languages.
  - High resource requirements.
  - Users with limited resources may suffer.

- **Piper** [29] a fast, local neural text to speech system that sounds great and is optimized for the Raspberry Pi 4. Piper is used in many projects like home assistant, open voice operating system and many.

Features:

- Support many languages like Arabic, English, Spanish and many.
- Providing a google colab to try before installing it on your PC.
- Ease to use and free.

Drawback:

- The quality of voice generation in some languages is not perfect, like Arabic language some words do not pronunciation as native or close to native speaker, also the voice generation it talks fast not like a normal voice.

- **So-vits-svc** [30] focuses on Singing Voice Conversion (SVC) rather than Text-to-Speech (TTS). But you can integrate other TTS easily.

Features:

- Open source.
- Ease to use.
- Providing a google colab to try before installing it on your PC.
- The quality of the voice generated is good compared to the samples (22 clips each 10s) we gave to the model to train.

Drawback:

- Limited in languages.
- The training time is bad as we mentioned in features the samples was not that much, but it took much time to generate the clone voice.

3.2.2 Text Annotation

- **LabelBox** [31] is a powerful platform for text annotation that facilitates the creation of high-quality training datasets for machine learning and artificial intelligence projects.

Features:

- LabelBox allows multiple users to work together on annotation projects, enabling teams to efficiently collaborate and review work in real time.

- The platform provides flexibility to design annotation workflows tailored to specific project requirements. Users can create custom labels as you see in Figures 22 and 23, set guidelines, and define review processes to ensure consistency and quality across annotations.

Drawbacks:

- LabelBox offers a free version; however, it imposes limitations on certain features. To gain access to advanced functionalities and increased usage limits, users are required to subscribe to a paid plan.

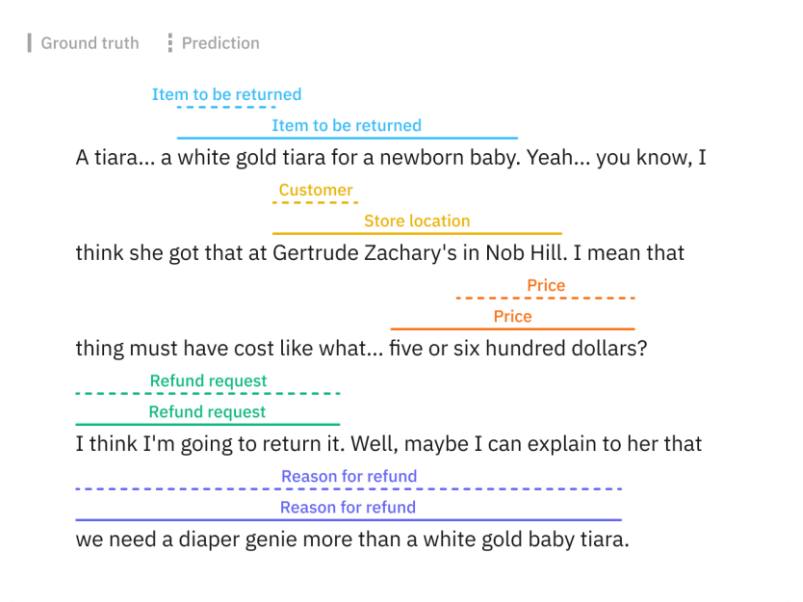

Figure 22: LabelBox [31]

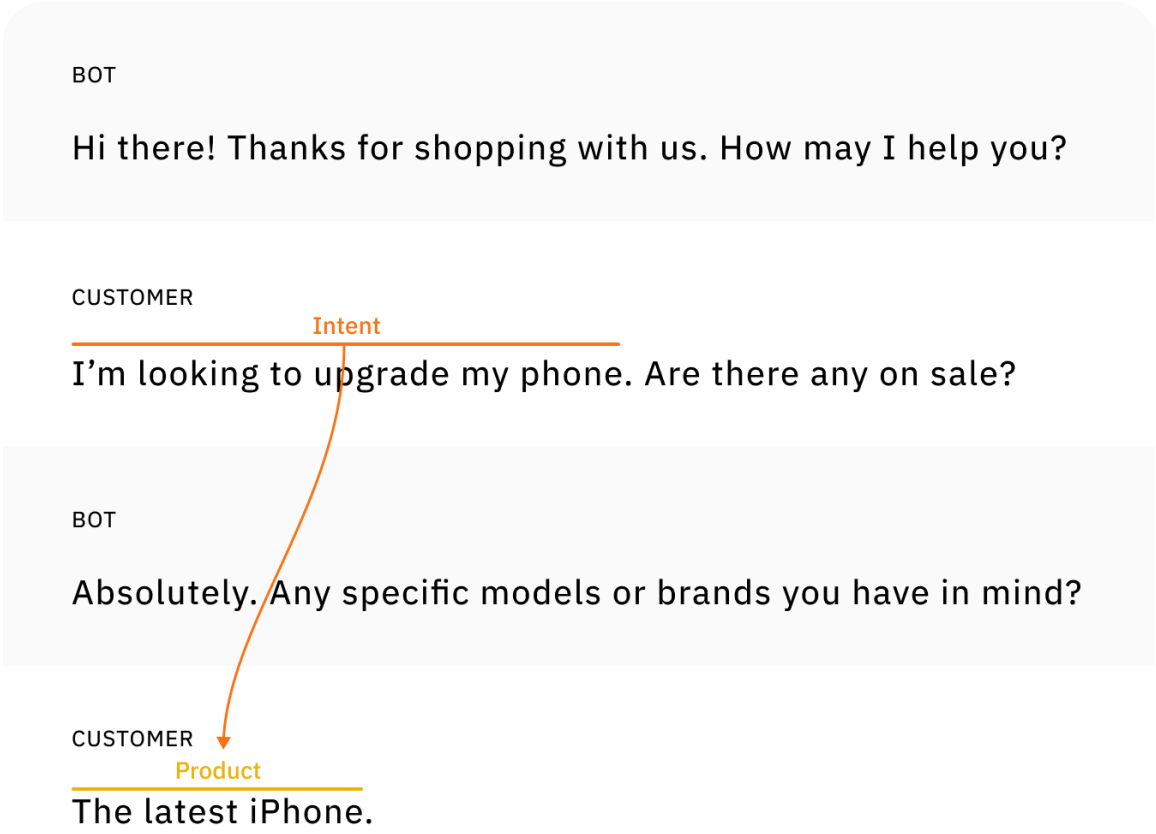

Figure 23: LabelBox [31]

- **spaCy** [32] is a powerful open-source library for Natural Language Processing (NLP) that supports efficient text annotation and provides tools to build robust machine learning models, as shown in Figure 24.

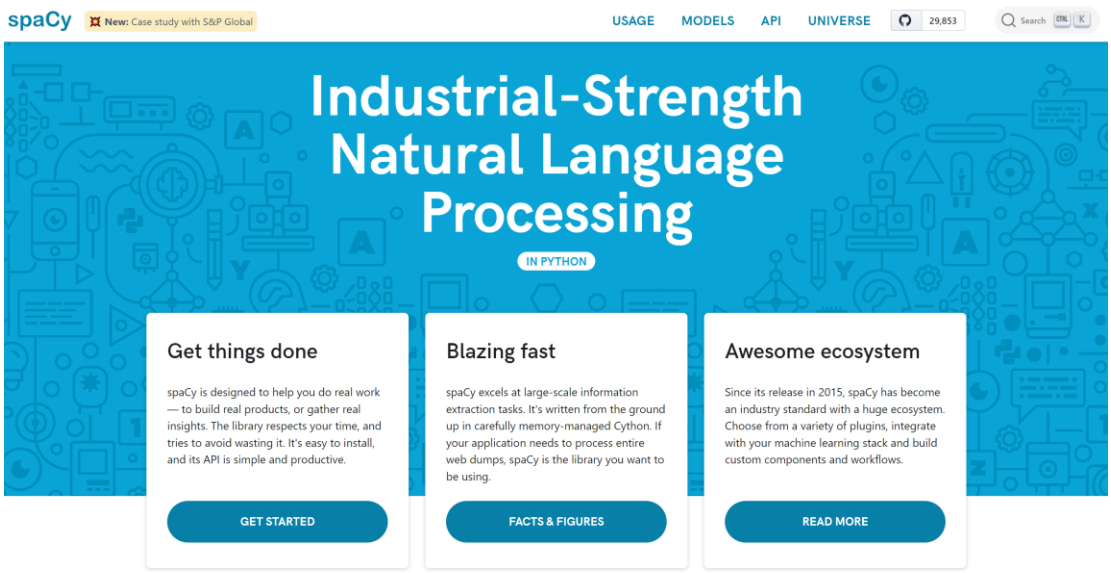

Figure 24: spaCy [32]

- **Flair** [33] is an open-source Natural Language Processing (NLP) framework designed to perform efficient text annotation, with a focus on simplicity and flexibility. Flair supports modern NLP tasks using state-of-the-art deep learning models, as shown in Figure 25.

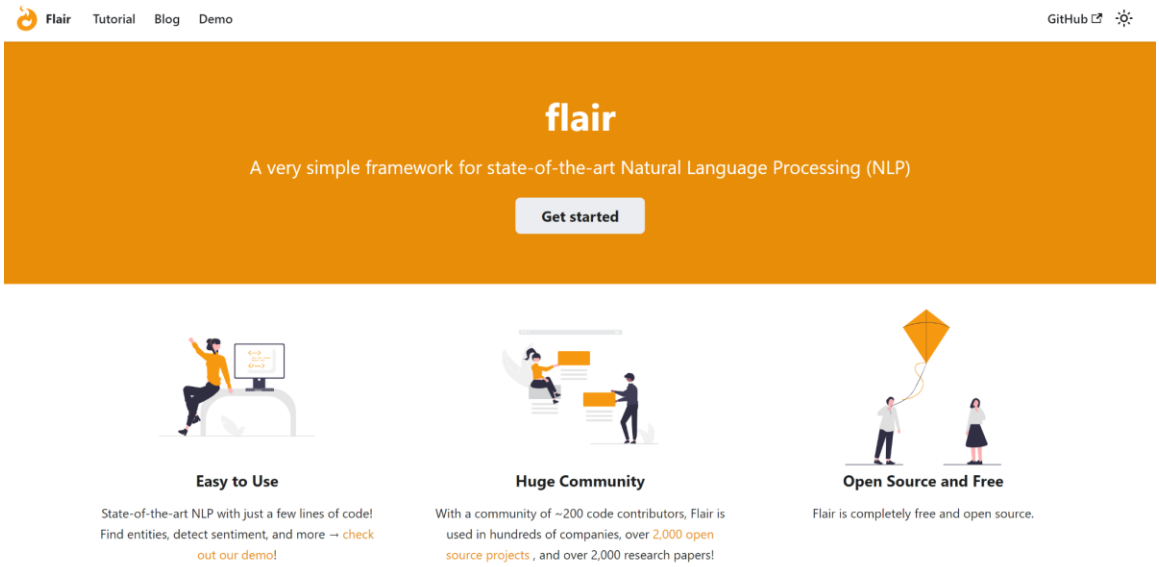

Figure 25: Flair [33]

- **StanfordNLP** (Stanza) [34] is a powerful NLP library developed by the Stanford NLP Group. It supports efficient text annotation across multiple languages, with a focus on providing high-quality, pre-trained models for various linguistic tasks, as illustrated in Figure 26.

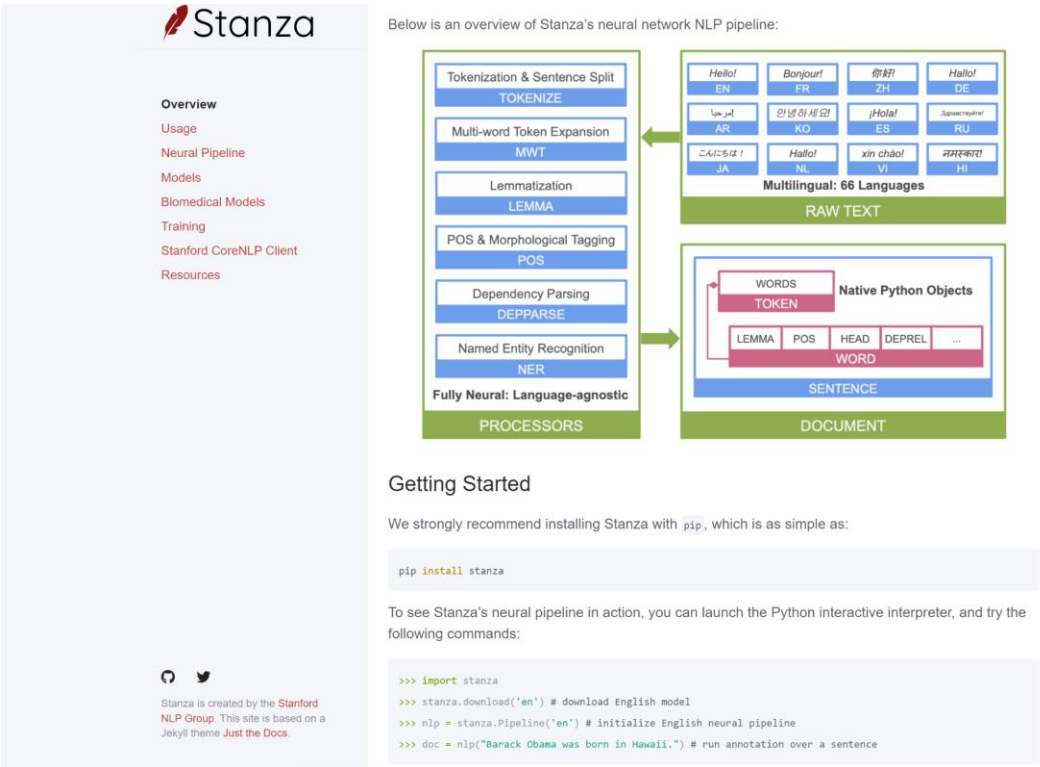

Figure 26: StanfordNLP (Stanza) [34]

After experimenting with spaCy, Flair, and StanfordNLP (Stanza), we identified several strengths and limitations of each tool based on our experience. Below is a summary of the key pros and cons we found, as illustrated in table 5.

spaCy:

- **Pros:**
  - Highly optimized for speed and recognized as one of the fastest NLP libraries.
  - Provides high-quality pipelines for multiple languages.
  - User-friendly for beginners and integrates well with other tools.
  - Excellent at efficient tokenization.
- **Cons:**
  - Primarily designed for production-level NLP tasks, with limited flexibility for research-level customizations.
  - Can be resource-intensive on RAM.
  - Might struggle with more complex datasets compared to other models.

Flair:

- **Pros:**
  - Known for achieving excellent results in tasks like part-of-speech (PoS) tagging.
  - Provides easy-to-use methods for custom training, especially for transfer learning.
- **Cons:**
  - It tends to be slower than spaCy due to its RNN-based architecture.
  - Memory-intensive because of the use of contextual embeddings.

StanfordNLP (Stanza):

- **Pros:**
  - Offers very high accuracy due to its deep learning models, often outperforming statistical models.
  - Optimized for researchers and developers in need of advanced models.
- **Cons:**
  - Although powerful, it can be more challenging to set up compared to spaCy or Flair.
  - Slower than spaCy, especially due to the complexity of its deep learning architecture.

Summary:

Through our hands-on testing of these tools, we found that each excels in specific areas. **spaCy** is ideal for fast, production-level NLP tasks, **Flair** offers advanced customization options through transfer learning, and **Stanza** is well-suited for researchers looking for high accuracy with multilingual support. However, each tool presents trade-offs in terms of speed, ease of use, and computational requirements, as shown in table 5 the comparison table.

Table 5: Comparison between models

	Pros	Cons
spaCy	- Highly optimized for speed. - Well-known for being one of the fastest NLP libraries. - Offers high-quality pipelines for multiple languages. - User-friendly even for beginners. - Intergrates well with other tools. - Efficient Tokenization.	- Mainly built for production-level NLP tasks, which doesn't really offer a level of customizations as research-focused libraries. - Can be resource-intensive on RAM. - Compared to the other two tagging models shown here, it might lag due to being untrained on more complex datasets.
Flair	- Known for achieving advanced results on tasks like PoS tagging. - Provides easy to use methods for custom training based on transfer learning.	- It tends to be slower than spaCy due to being RNN-based. - Could be memory-intensive due to the usage of contextual embeddings.
StanfordNLP (Stanza)	- Could achieve very high accuracy, because it's based on deep learning methods, and it often outperforms statistical models. - Its architecture and design are optimized for both researchers and developers in need of high-tech models.	- Even if powerful, usage and set-up could be harder than Flair or spaCy, especially for beginners. - Could be slower than spaCy due to the complexity of its deep learning models.

3.3 Discussion

This section discusses some of the drawbacks of the reviewed articles as well as the features they provide and how it's related to our project and how this project aims to tackle them. However, the research studies are successful in defining models like Tacotron2 and how it's accuracy for synthesizing the voice.

Although the existing research and software on voice cloning and text annotation reflect significant advancements, several challenges remain, which our project addresses them. Traditional voice cloning approaches such as Tacotron2 and Transformer-based TTS models have made progress in generating high-quality synthetic speech. However, they frequently encounter constraints such as the requirement for extensive datasets, slow training, and the inability to handle emotional expression or multilingual scenarios effectively. Additionally, speaker adaptation techniques, while effective, require substantial fine-tuning and data resources to create personalized voice clones.

This project aims to solve these challenges by combining pre-trained models with designated tags, enabling faster voice generation without requiring extensive fine-tuning. Furthermore, solutions like Resemble AI focus on emotional expression, but you need to have subscription plans to use it. Our approach offers an open and customizable platform for Quranic translations, reducing the dependency on costly commercial services.

Text annotation tools like DoTAT, TeamTat, and FITAnnotator highlight the significance of organized tagging frameworks for effective annotation processes. These tools highlight the role of collaborative workflows and domain-specific tagging for enhanced annotation accuracy. However, most existing systems are oriented toward general NLP tasks, not specialized use cases like Quranic recitation. Our project solves this gap by producing contextual tags tailored for Quranic audio, such as pronunciation emphasis on specific words and pauses to reflect the proper reading style. This solution ensures accurate pronunciation while also allowing the dynamic update of translations.

So, the main objective of this project is to aim to build on producing a well fine tune for Quran translations and the way it pronounces these words. We plan to investigate several steps of training a model to find the most suitable model.

This system will offer the most extensive solution by providing a combination of the latest voice cloning technology and dynamic text annotation system. This project will make the system tailer made for Quranic audio generation. Future development of this project may focus on increasing customizability for voice parameters, adding more options for voices and addition of more languages. This will make it reach a bigger audience and have a more global reach.

Chapter 4: System Analysis

4.1 System Requirements

The system requirements are divided into functional and non-functional requirements, which explains the functionality of the overall system.

4.1.1 Functional Requirements

- The user should be able to create an account.

- The user should be able to login and logout whenever he/she wants.

- The user should be able to edit their own profile.

- The user should be able to add a tag on notable words.

- The user should be able to choose a voice to clone from the voices in the system.

- The user should be able to download the generated audio file.

- The user should be able to choose any sura.

- The user should be able to choose the project that he/she will work on it.

- The user should be able to choose between chapters of sura.

- The user should be able to stop playing the voice.

- The user should be able to choose the translation.

- The user should be able to highlight the words or space or lines or the entire chapter in text box.

- The user should be able to repeat the voice.

- The user should be able to distinguish between chapters that he/she completes and that still not finished.

- The administrator should be able to login and logout at any time.

- The administrator should be able to add more voice options.

- The administrator should be able to add more versions of the translated Quran.

- The administrator should be able to add more languages to the system.

- The administrator should be able to update the models.

- The system should be able to save the work of the user to his/her account.

- The system should send a verification code to the user.

- The system should reject bad quality audio.

- The system should allow tagging in multiple languages.

- The system should display a message when audio gets rejected.

- The model should be able to generate audio in multiple languages.

4.1.2 Non-Functional Requirements

- The model should ensure to be fast at playing the voice.

- The model should be very accurate at pronouncing words.

- The model will take less than a minute to generate the new modified pronunciation and/or emotions.

- The system should be able to save users data in the database.

- The system should be very secure.

- The system should be excellent at error-handling.

- The system should have a user-friendly interface.

- The system should be able to run at all times.

- The system should encrypt user's data.

- The system shall be available 24/7+.

- The system should be compatible with major web browsers (e.g., Chrome, Firefox, Safari).

- The system will support (Windows, Linux, MacOS, iOS, and Android).

- Python programming language will be used to implement the models.

- HTML, CSS, and JavaScript will be the stack to develop the website.

4.2 Actor Goal List

Table 6 shows how the actors interact with the system to reach their goals.

Table 6: Actor Goal List

Actor	**Goal**
User	• Sign-up • Log-in. • Log-out. • Edit profile. • Choose project. • Choose the cloned voice. • Choose translation. • Choose space or words to tag them. • Play the sound.
Model	• Integrate voice cloning. • Training and updating. • Process input text. • Handle multiple languages.
Administrator	• Add voice. • Login. • Logout. • Add translated version of Quran. • Add language. • Update model.
Cloud-server	• Send data to dashboard.

Chapter 5: System Design

In this chapter, we will illustrate our system's functionalities, architecture and the relationship between different objects of the system and how they interact with each other. It also provides illustrations of the interface prototype for the web application of the system. Also, it will provide an illustration of the model algorithm, and its training process.

5.1 System use-cases

A use-case diagram depicts the interactions between the system and the external actors [35]. Figure 27 shows the actors interacting with their system functionalities.

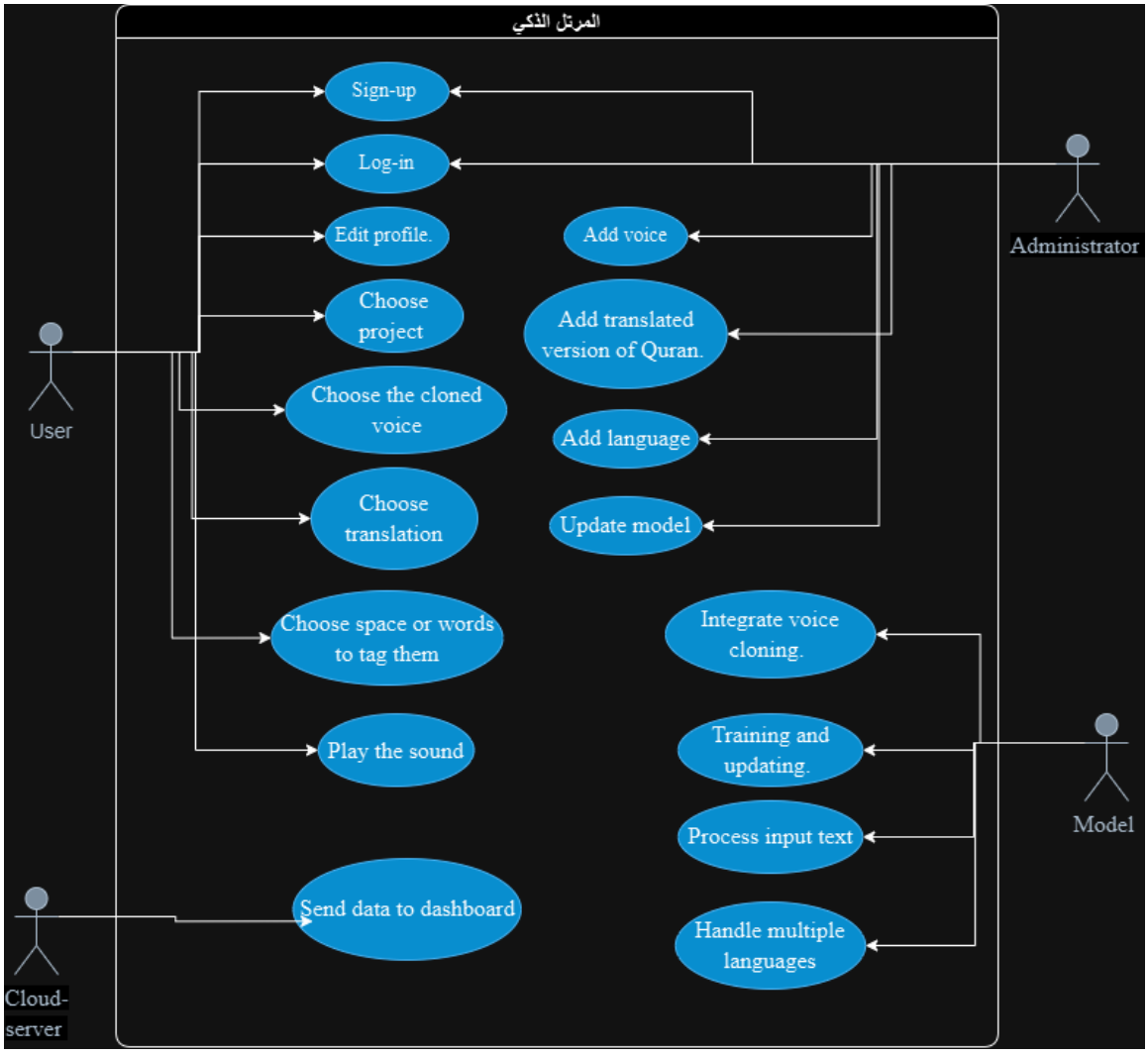

Figure 27:Use-case diagram

5.2 Interaction Diagrams

Interaction diagrams are a graphic representation of a group of objects interacting in a typical use case. This can involve multiple use-case scenarios or a single use-case. The two main types of interaction diagrams are a sequence diagram and a collaboration diagram. The sequence diagram shows how objects interact in timeline while a collaboration diagram shows the organization of objects in a flow.

Figure 28 shows the process of the user login, assuming he already has an account. The user first enters his/her username and password via the GUI, and the GUI communicates it to the system, which then verifies the information. If the login is successful, the system will redirect the user to the dashboard. The user has five attempts to enter the correct username and password, or the login ability of the user will be temporarily shut down.

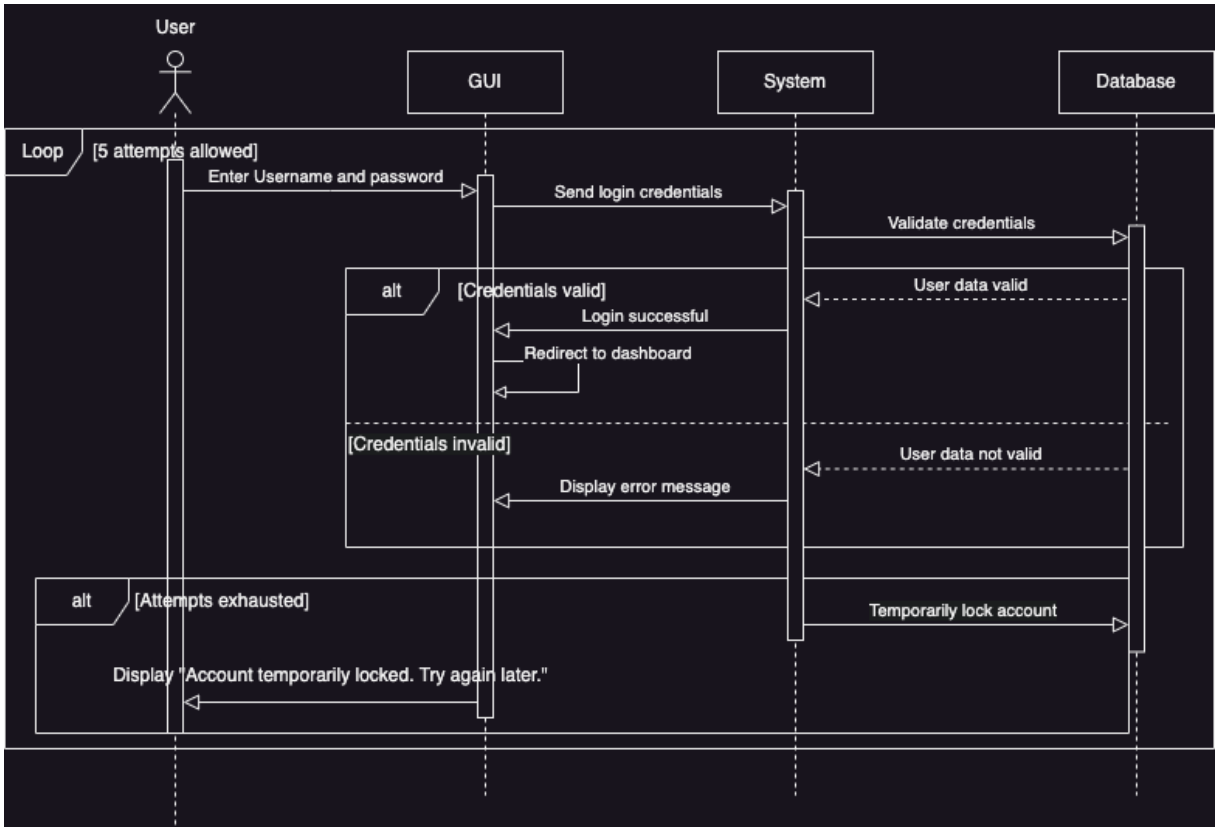

Figure 28:Sequence diagram: Login

Figure 29 shows how the user creates a project. Assuming the user is already logged in, the user opens the project window via the GUI, which will display the already created projects from the system, retrieved from the database. The user then chooses the "Create Project" option, which will display the project creation options, and the user fills in the details. If the details are valid, the project will be created; otherwise, it will display an error message that involves the failed creation of the project.

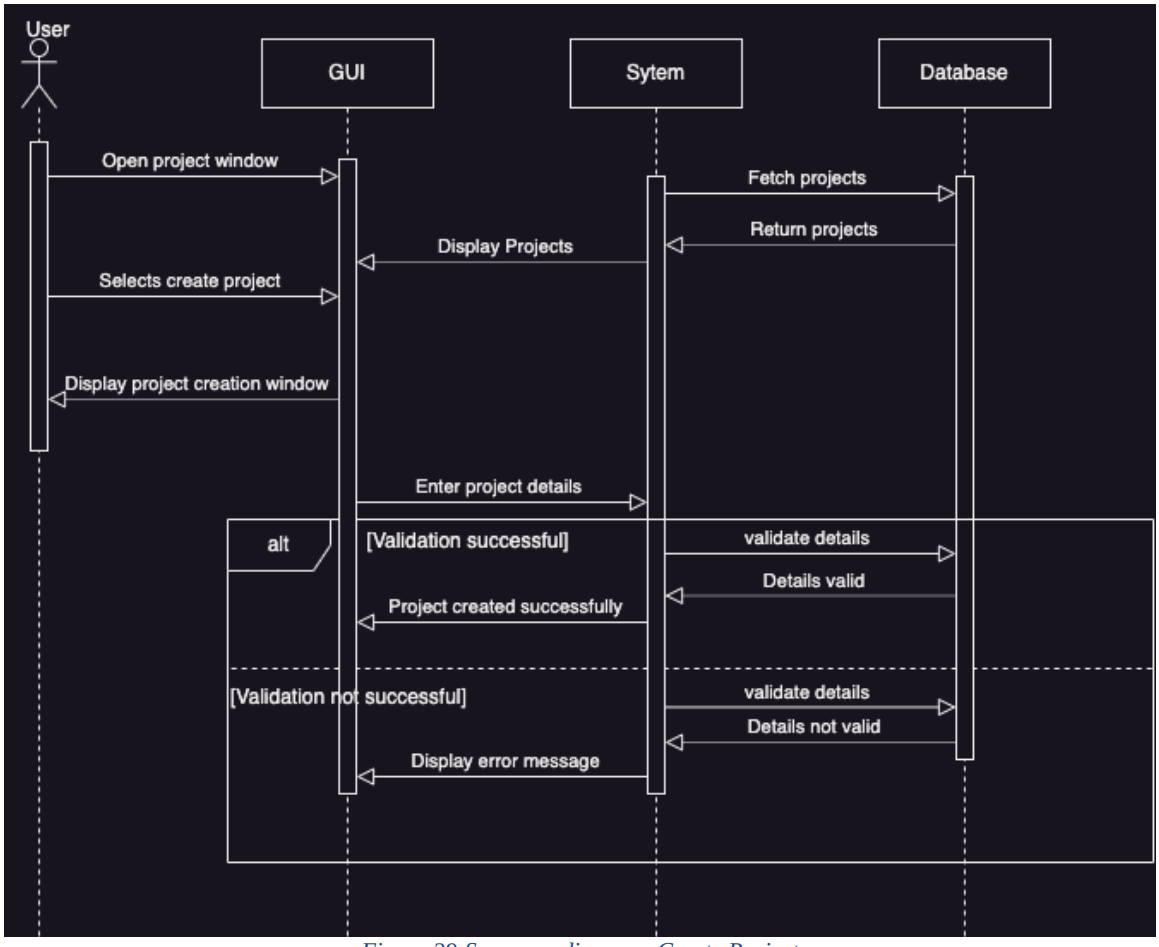

Figure 29:Sequence diagram: Create Project

The third sequence diagram shown in Figure 30 shows the steps of generating an audio file. The user, after selecting the translated version of the Quran, will be asked by the system to choose the sura. After it is selected, the system displays the translated version of the chosen sura. The user then selects the voice from the ones given by the system; if there is a need for adding annotations, the voice cloning engine will generate the file with the annotations in consideration and return it to the user. Otherwise, the generated file will be returned generated without any guidelines given by annotations.

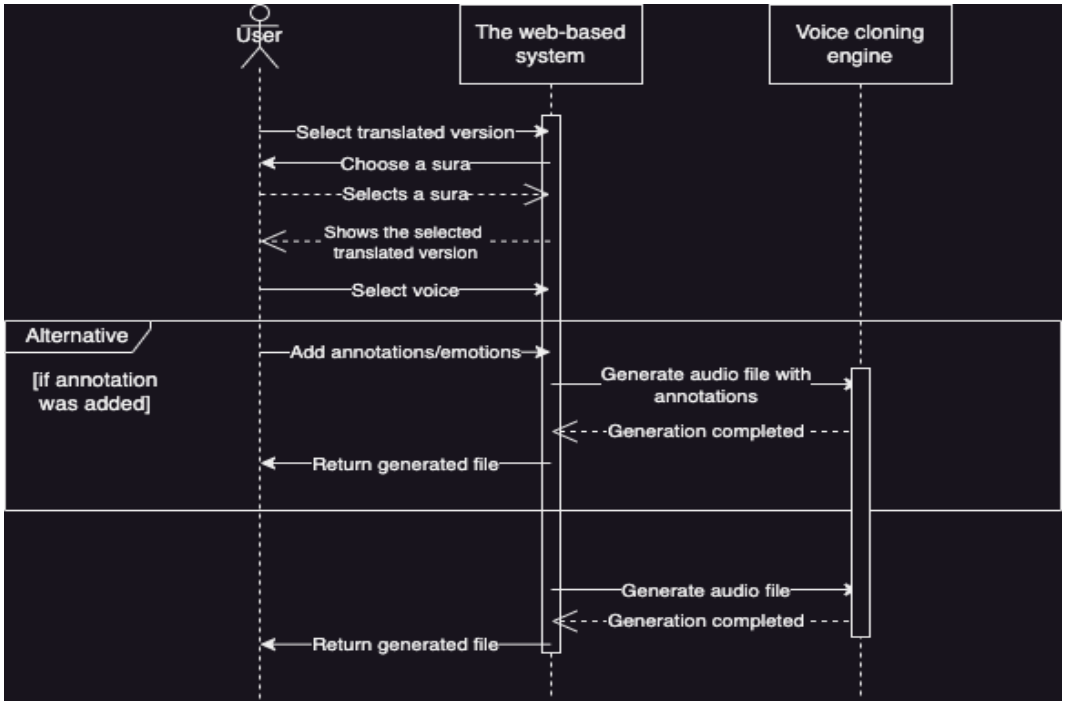

Figure 30:Sequence diagram: Generate Audio File

5.3 Class Diagram

Class diagram is a type of diagram and part of UML that defines structure of system in terms of class, attributes and methods, also provides the relationship between different classes [36].

Classes:

Profile: This is an abstract class since administrator and user class have common functions and attributes.

User: This class provides fields to store user information and the functions used by the user.

Administrator: This class provides functions that administrators need to perform.

Project: In this class it will contain all project information and functions that will be used on the project.

Audio: This class provides us with information about audio.

Text: This class provides us information about text.

Relationships between the classes:

User & Administrator: have an inheritance relationship with Profile class since both classes have common functions and attributes.

Project & User: It's a one-to-many relationship, so that each user can have multiple projects, and each project belongs to one user.

Project & Audio: It's a one-to-many relationship, so one project can have multiple audios which can be used for cloning and generating.

Project & Text: It's a one-to-many relationship, each project has multiple texts, and each text belongs to a project which can be used for text Annotation.

After we saw the classes and the relationships between them, Figure 31 shows the class diagram.

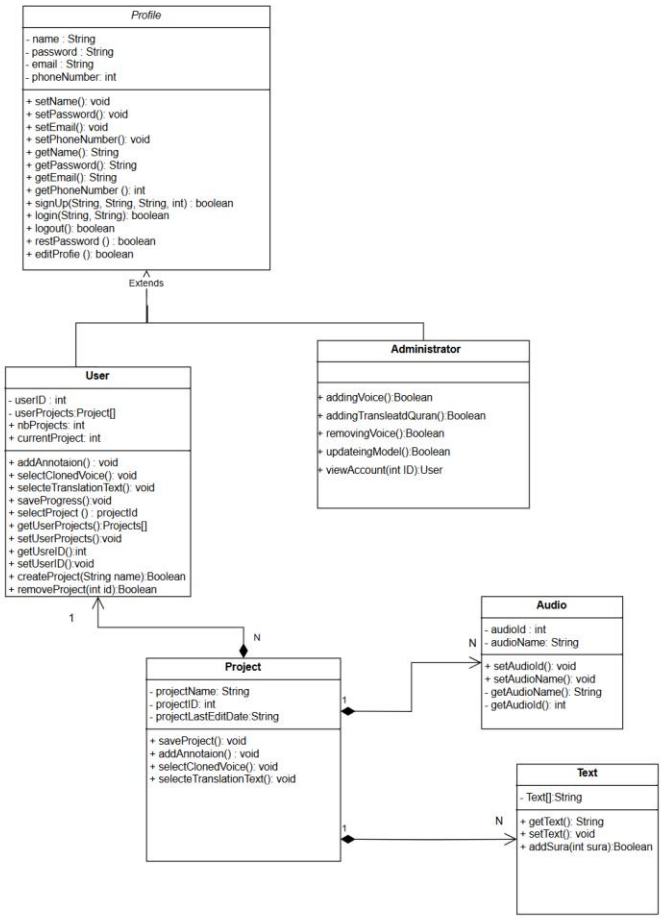

Figure 31: Class Diagram

5.4 System Architecture

In this section we will explain and show the system architecture. System architecture is how our software interacts between its components.

In Figure 32 we see that our system has three main comments, and they are:

- **Website**

  The website sends and receives data from cloud server through API, so the user may log in successfully and can choose between his projects or create new one so he can choose the translated version of Quran and choose any sura so he can choose any chapter and do tagging or generate voice from it or both.

- **Cloud server**

  The cloud server contains the Database that contains data the required by web dashboard and website, Model that can clone voice and generate voices and Text annotation that can tag the text and provide a predefined tag.

- **Admin Dashboard**

  The admin dashboard is connected to the cloud server via API, to receive information about the users and the projects done by this website.

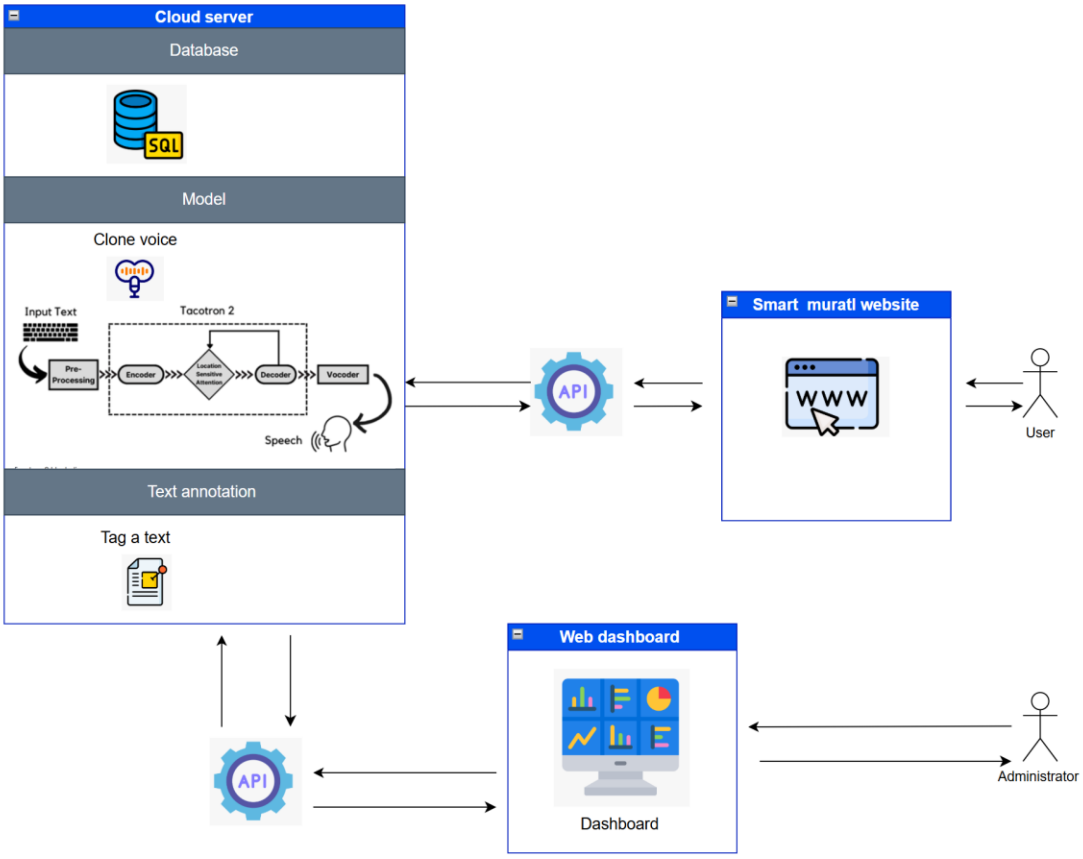

Figure 32:System Architecture

5.5 Database Design

5.5.1 ER Diagram

Figure 33 shows the entity relationship diagram. It's a simple diagram that shows each entity's attributes and the relationship with other entities.

Entities:

Users (user_id, username, email, password_hash, is_admin, created_at)

Projects (Project_id, user_id, name)

QuranVersions (version_id, name, language, description, created_at)

Surahs (surah_id, surah_number, name, arabic_name, number_of_ayahs)

Verses (verse_id, surah_id, verse_number, text, is_altered)

Tags (tag_id, name, description)

VerseTags (verse_tag_id, verse_id, tag_id, start_word_index, end_word_index, created_at)

Voices (voice_id, name, description, file_path, created_at)

AudioRequests (request_id, Project_id, user_id, status, audio_file_path, requested_at, completed_at, start_verse, end_verse, include_tags)

Relationships:

Users & Projects: it's a one-to-many relationship. One User can create many projects.

QuranVersions & Surahs: It's a one-to-many relationship. So that one QuranVersions can contain many Surahs.

Surahs & Verses: It's a one-to-many relationship: One Surah contains many Verses.

Verses & VerseTags: It's a one-to-many relationship. One verse can have many versetags.

Tag & VerseTags: It's one-to- many relationship. Each tag is associated with many Verses.

Projects & VerseTags: It's a one-to-many relationship. A project can apply many tags on the verse.

Projects & Voices: It's a one-to-many relationship. A voice can be in many projects.

Projects & AudioRequests: It's a one-to-many relationship. A project can have many audio requests.

Projects & Quranversions: It's a one-to-one relationship. One project can have one quran version.

Surahs & Quranversions: It's a one-to-many relationship. A quran version can have many surahs.

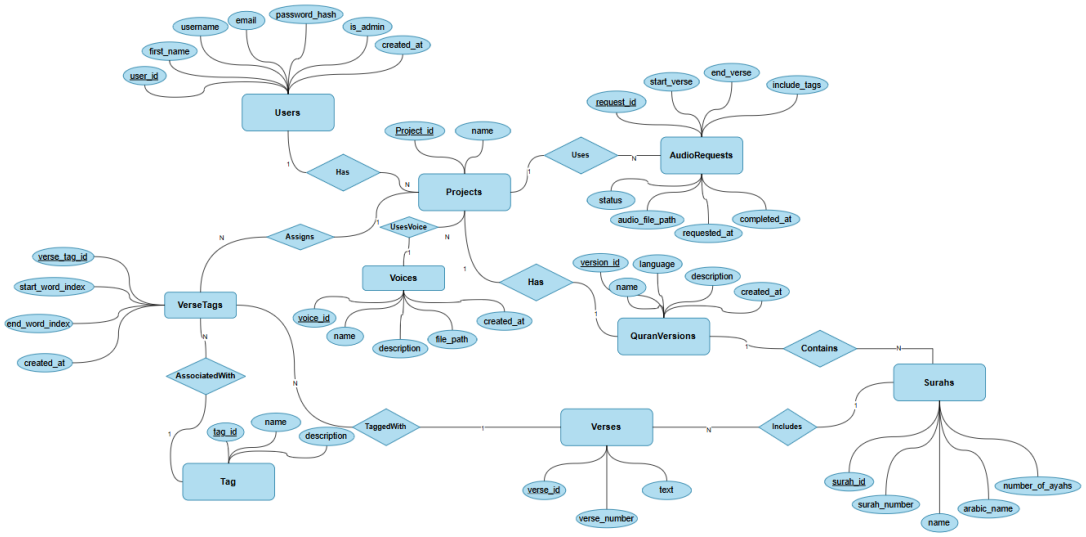

Figure 33:ER Diagram

5.5.2 Relational Database Schema

Figure 34 shows the Relational Database Schema of the system.

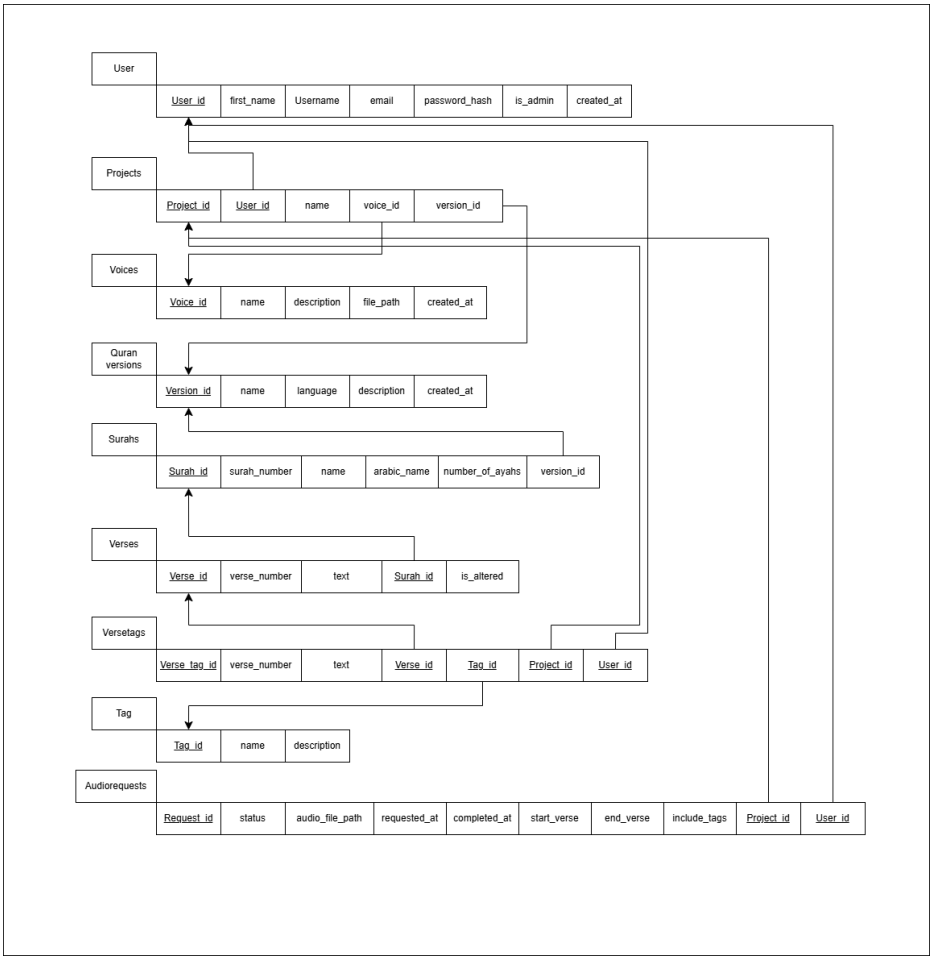

Figure 34: Relational database schema

5.6 User Interface Prototype

In this section we show the overall user interface of the web application. Figure 35 shows the user login page.

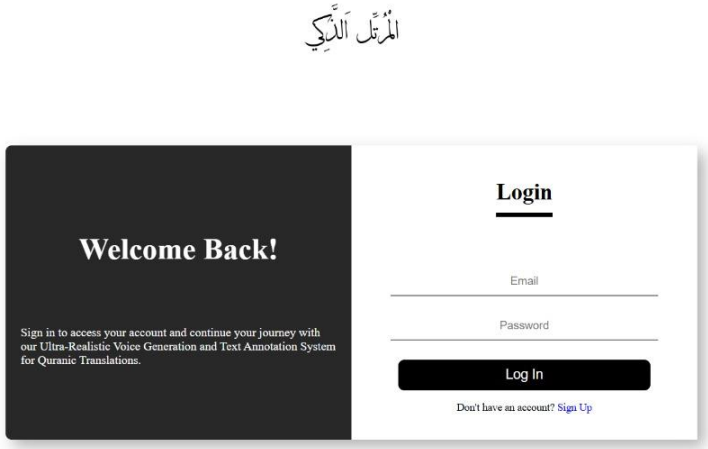

Figure 35: Login page

Figure 36 illustrates the Home page interface, allowing users to create a new project or access and manage their existing projects seamlessly.

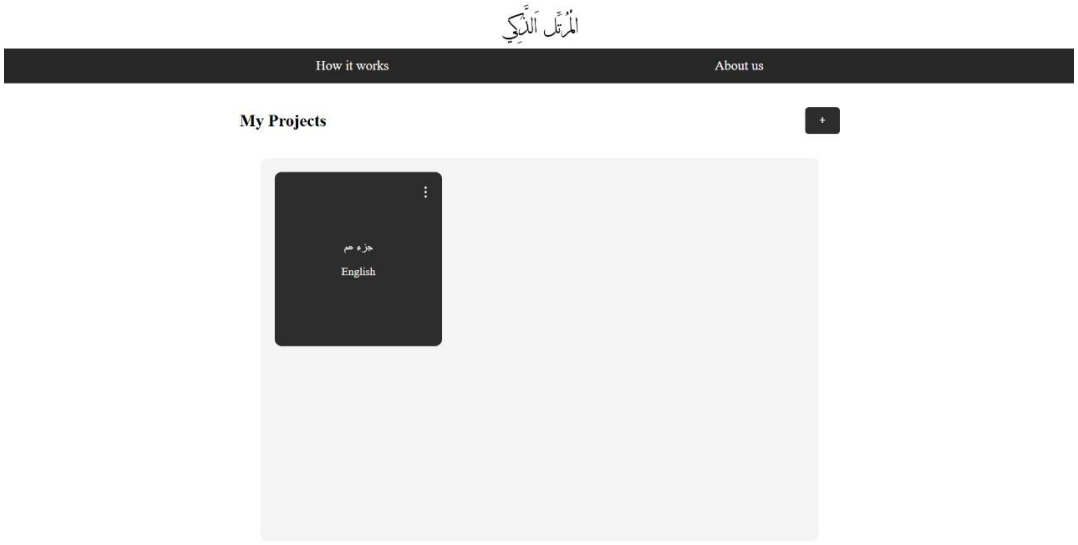

Figure 36:Home page

Figure 37 illustrates the interface for adding a new project, where the user can enter the project name, select the translated version, and specify the language to create a customized project seamlessly.

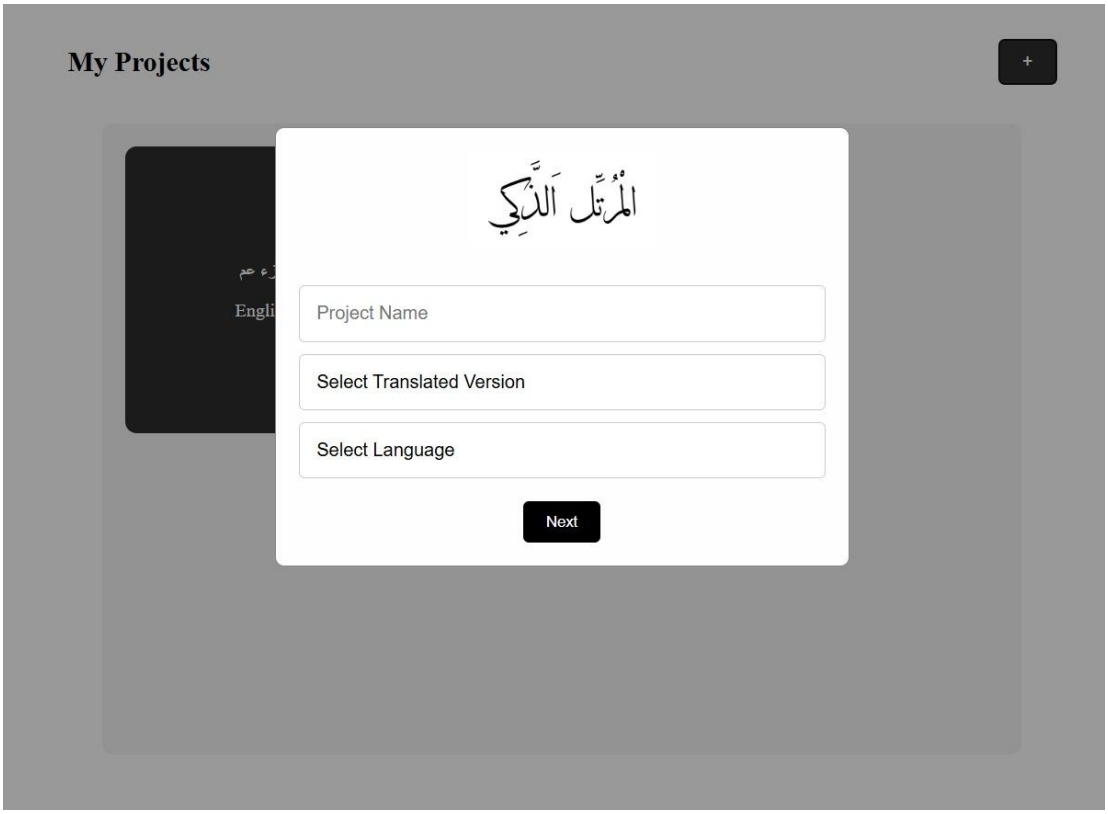

Figure 37:Add new project page

Figure 38 illustrates the main page of the application, where the user can select one of the cloned voices provided by the system. Additionally, the user can choose any Surah from the Quran, apply specific emotions and emphases to the recitation, and export the audio file for further use. This interface offers an intuitive and interactive experience, allowing for precise customization of the audio output.

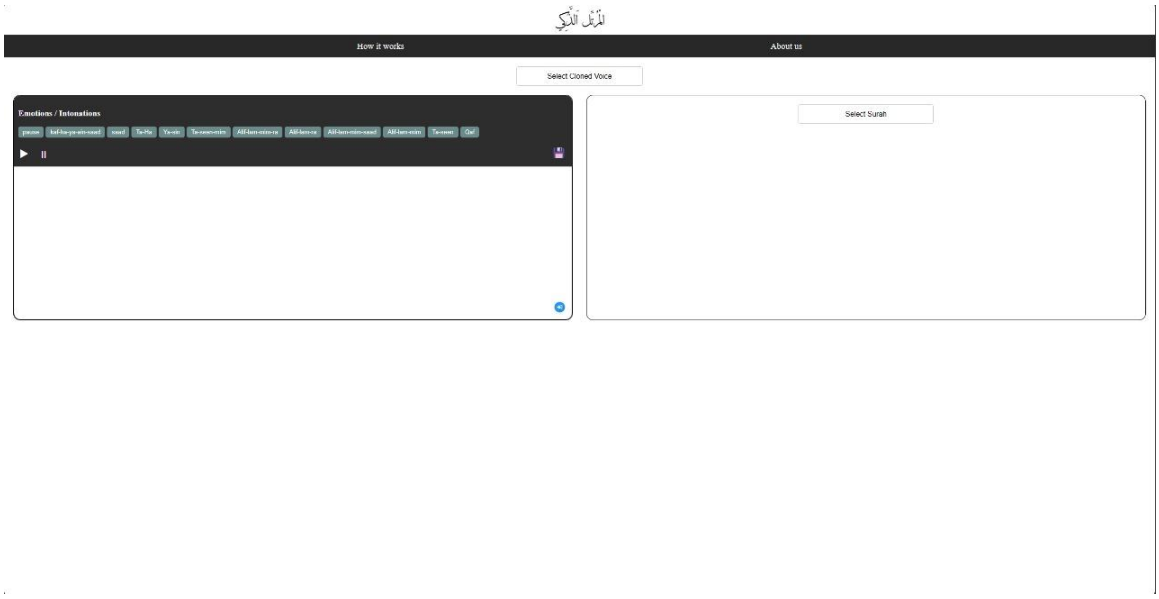

Figure 38:Main page

5.7 Algorithms

In this section we will explain the algorithms of voice cloning and text annotation that we used in our project.

5.7.1 Voice cloning

In this section we will give an overview of the voice cloning algorithms we used.

Tacotron2 [37] is an algorithm in text-to-speech which is considered one of the best among the other algorithms in text-to-speech. Tacotron2 uses a recurrent sequence-to-sequence feature prediction network with attention which predicts a sequence of mel-spectrogram frames from an input character sequence. The reasons for choosing Tacotron2 as a starting point is: (1) High quality speech synthesis which is cable of generating speech with high naturalness and intelligibility. (2) It directly maps text input to a mel-spectrogram using an encoder–decoder with location-sensitive attention. (3) Attention mechanism which gives smooth and accurate for difficult pronunciations. Figure 39 shows the pipeline of the algorithm.

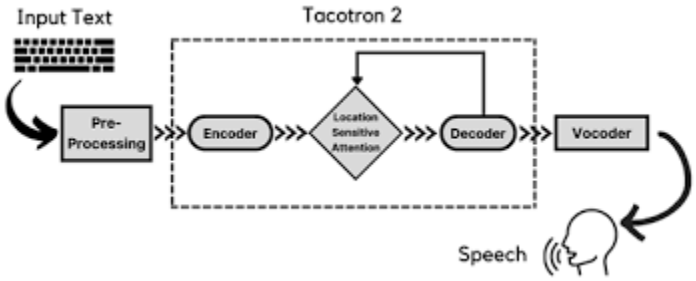

Figure 39:Pipline of Tacotron2 [38]

StyleTTS2 [39] is a text to speech model that "leverages style diffusion and adversarial training with large speech language models to achieve human-level TTS synthesis." StyleTTS2 is different from its predecessor because it models styles as a latent random variable through diffusion models, generating the most suitable style for the text without requiring reference speech. Our reason for choosing StyleTTS2 is the fact that, in the end, it produces output faster than the other models we tried.

F5-TTS [40] is a "fully non-autoregressive text-to-speech system based on flow matching with Diffusion Transformer (DiT)." Even though its quality is on par with StyleTTS2, it's still slower. Furthermore, StyleTTS2 is better at maintaining natural prosody and style transfer. As seen in Figure 40, input text gets converted to a character sequence, which is padded with filler tokens to match the input speech's length. Then, it's refined by ConvNeXT blocks [41]. Our reason for choosing it as part of our options is because it's one of the faster models we've found, but it's still slower in producing the output than StyleTTS2.

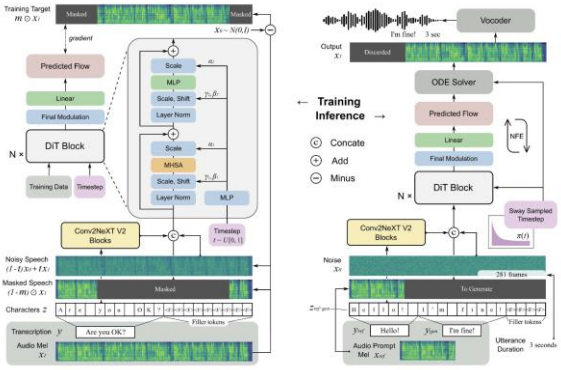

Figure 40: F5-TTS training overview [41]

Tortoise-TSS [42] is an "expressive, multi-voice text-to-speech system", an original evaluation suite was created for it which used CLVP to produce a distance metric between real samples and generated samples. An open source wav2vec model was also used to characterize the “intelligibility” of a speech segment. However, in comparison to StyleTTS2 and F5-TTS, it still falls off our demands and expectations.

5.7.2 Text Annotation

For text annotation, we will be using spaCy. spaCy uses a mix of rule-based statistical and deep learning models. spaCy uses a pipeline approach where text is sent through a pipeline which consists of spaCy’s own tagging system and both custom built pipelines or modified pipelines [43], displayed in Figure 41.

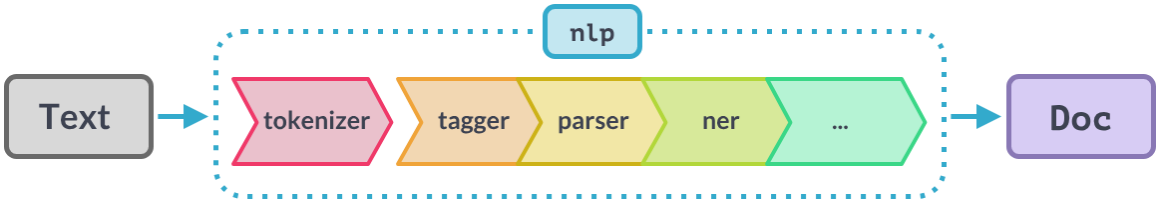

Figure 41: spaCy pipeline [43].

5.7.3 Model Training

In this section we explain how the model will be trained and show the tasks of this process. First, it starts with preprocessing the dataset voices and the transcript voices. Second, we take the preprocessing dataset to be the input to a model that initialized the training process. One example of model training is the StyleTTS2 training process which will directly generate mel-spectrogram from given input text then the vocoder will generate a voice from mel-spectrogram [44]. Figure 42 shows the flowchart for the training process.

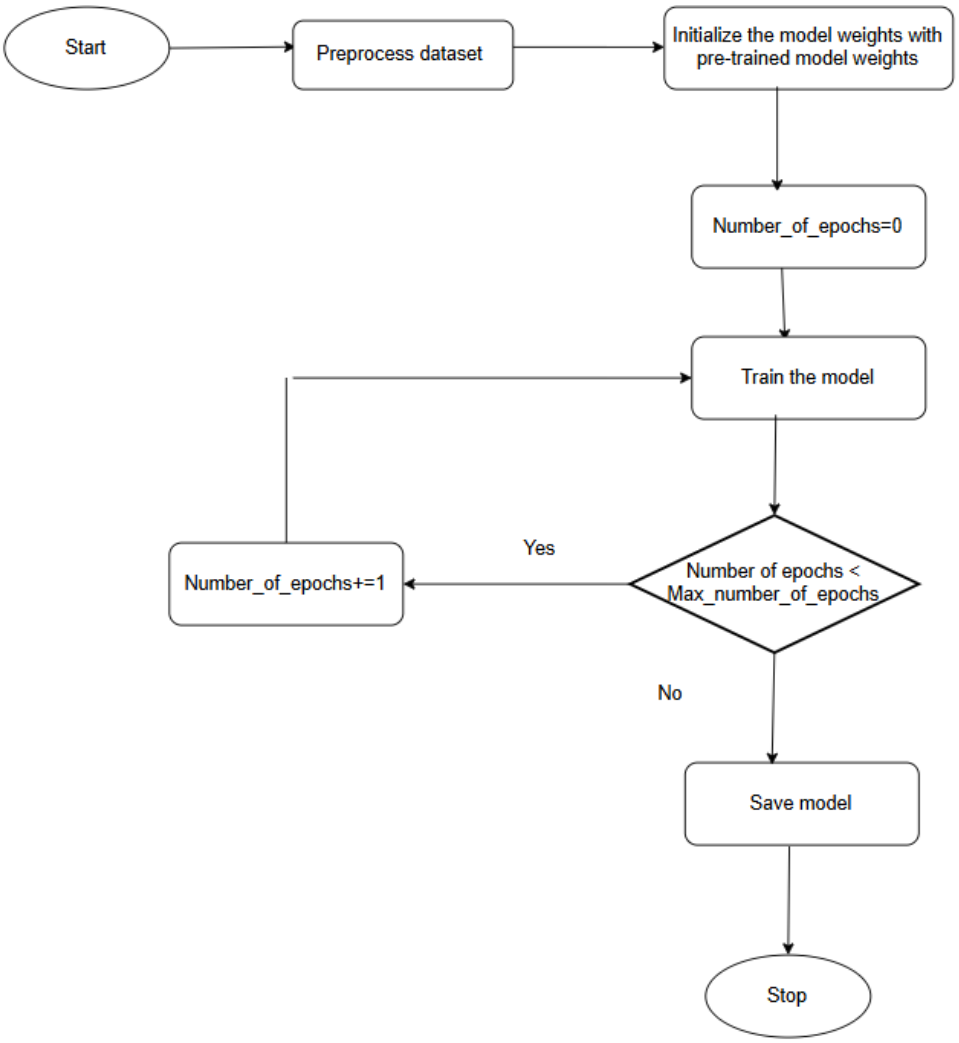

Figure 42: Flowchart of training process.

Chapter 6: System Implementation

In this chapter, we will discuss the system implementation including the back-end and front-end development, the models tested in action, and their respective results.

6.1 Back-End Development

In this section, we will discuss the inner work of the back-end of our system. The models which we tested for their implementation, how we assessed them, how we were able to connect our MySQL database with our website's UI, which allowed the user's input, projects, selected language versions of Quran, and more to be saved into the database.

6.1.1 Voice Cloning Models

Figure 43 presents a concise Python script that orchestrates the complete inference workflow for StyleTTS2, a diffusion-based text-to-speech model. The pipeline begins by parsing all model hyperparameters from a YAML configuration file and enforcing deterministic behavior via fixed random seeds. Depending on hardware availability, it then instantiates the neural architecture on either a CUDA-enabled GPU or CPU. Input preprocessing comprises a text-cleaner module and an Espeak-based phonemizer, which together convert raw text into stress-annotated phonetic sequences, while a MelSpectrogram transform prepares any reference audio. For each specified reference waveform, the script reloads the corresponding model checkpoint, computes a low-dimensional style embedding, and executes the diffusion sampler over the phonetic input. Key sampling parameters (α, β, and the number of diffusion steps) are exposed for systematic tuning of the fidelity-latency trade-off. Finally, the resulting waveform is exported as a 24 kHz WAV file, and a real-time factor metric is reported to quantify inference efficiency. An optional secondary-checkpoint comparison enables direct evaluation of alternative model snapshots.

```python
def main():
    set_seeds()

    config = load_configurations(r"C:\Users\kking\Desktop\Learning Web\StyleTTS-WebUI\training\Michael\Michael_config.yml")
    device = 'cuda' if torch.cuda.is_available() else 'cpu'

    model_1 = r"C:\Users\kking\Desktop\Learning Web\StyleTTS-WebUI\training\Michael\models\epoch_2nd_00097.pth"
    model_2 = r""
    comparison = False

    model, model_params = load_models(config, device)

    global global_phonemizer
    global_phonemizer = phonemizer.backend.EspeakBackend(language='en-us', preserve_punctuation=True, with_stress=True)

    sampler = create_sampler(model)
    textclenaer = TextCleaner()
    to_mel = torchaudio.transforms.MelSpectrogram(
        n_mels=80, n_fft=2048, win_length=1200, hop_length=300)
    mean, std = -4, 4

    text = 'In The Name of Allah, The All-Merciful, The Ever-Merciful.Praise be to Allah, The Lord of the worlds.'
    reference_dicts = {'696_92939': r"C:\Users\kking\Desktop\Learning Web\StyleTTS-WebUI\voices\Michael\file___22_file___22_segment_2.wav"}
    # willa 28

    noise = torch.randn(1, 1, 256).to(device)
    start = time.time()
    print(device)
    for k, path in reference_dicts.items():
        load_pretrained_model(model, model_path=model_1)
        ref_s = compute_style(path, model, to_mel, mean, std, device)

        wav1 = inference(text, ref_s, model, sampler, textclenaer, to_mel, device, model_params, alpha=0.1, beta=0.6, diffusion_steps=30, embedding_scale=1.0)
        rtf = (time.time() - start)
        print(f"RTF = {rtf:5f}")
        print(f"{k} Synthesized:")
        from scipy.io.wavfile import write
        write(f"{k}_synthesized.wav", 24000, wav1)

        if comparison:
            load_pretrained_model(model, model_path=model_2)
            ref_s = compute_style(path, model, to_mel, mean, std, device)
            wav2 = inference(text, ref_s, model, sampler, textclenaer, to_mel, device, model_params, alpha=0.3, beta=0.7, diffusion_steps=100, embedding_scale=1.0)
            write(f"{k}_synthesized3.wav", 24000, wav2)
```

Figure 43: Python script for StyleTTS2's inference workflow

In project phase 1, Tacotron2 was our chosen model. But, in project phase 2, we looked for and tested other models that could work as close or better to what we hoped. And we found three notable ones which we tested and integrated:

- **StyleTTS2:** A model which "leverages style diffusion and adversarial training with large speech language models (SLMs) to achieve human-level TTS synthesis." [39]
- **F5-TTS:** A model which "utilizes deep learning algorithms to produce high-quality, human-like speech" and using "minimal audio input—often requiring as little as 10 seconds." [45]
- **Tortoise-TTS:** This text-to-speech model, as opposed to the previous two, claims to be built with one of its two priorities as "highly realistic prosody and intonation" [46]

These models, in both implementation and behavior, varied in their voice cloning outputs. And as we'll see in the next chapter, all models (including Tacotron2) were tested with a number of evaluation metrics, giving us a general indicator of which model we think is optimal.

6.1.2 Tagging

In project phase 1, we focused on developing spaCy, which we deemed was optimal. But, in project phase 2, we thought it'd be a better approach to preprocess the dataset instead. We started by preprocessing our dataset to prepare it for training. As part of this step, we manually added special tags—such as <Ta-Ha>—directly into the text. These tags were used to mark specific words or phrases that carry important meaning or pronunciation patterns. By training the model on this annotated data, it learned to recognize and correctly handle these tags during audio generation. This improved the model's ability to produce more accurate and expressive speech.

6.1.3 Database Iterations

After our initial phase in the project, the database we've created seemed more obsolete compared to what we aspired. There were a few problems during the development of our website, and many notes were taken to ensure that the database was more architecturally correct.

Figure 44 shows our first iteration of the database. At the beginning, we had the tables "quranversions", "versetags", and "voices" be in a non-identifying relationship with "user"; meaning that they wouldn't have to depend on the existence of their user to be identified themselves. "Projects", however, was in an identifying relationship, which would mean it wouldn't be defined if the user themselves weren't defined. The "quranversions" table was also dependent on "surahs", and "surahs" was also dependent on "Projects".

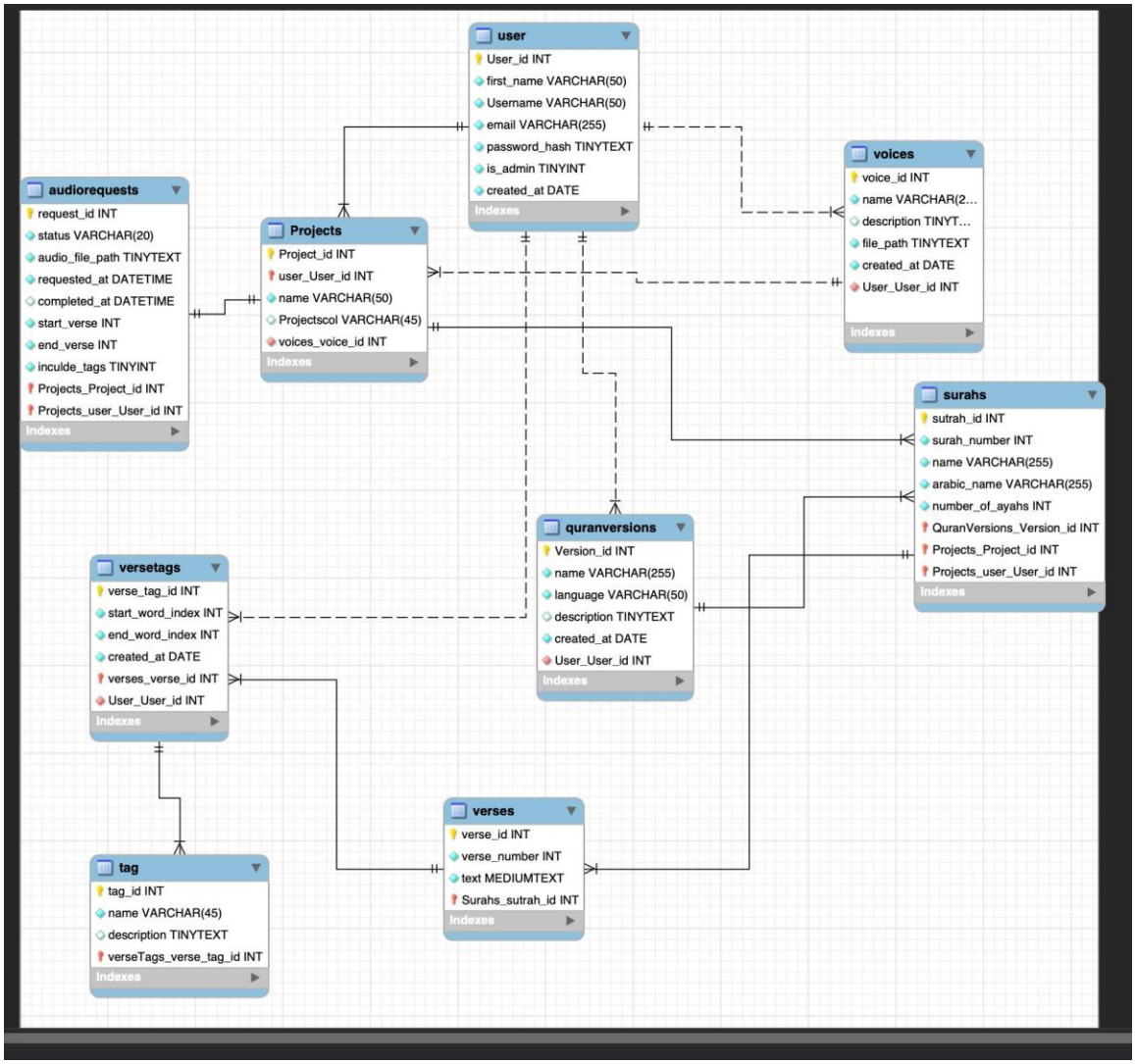

Figure 44: First iteration of our database

Except for the dependency between "Projects" and "user", notes in the previous figure had to be scrapped. Figure 45 shows the latest iteration of our database. We figured it'd make more sense to have "voices", "quranversions", and "versetags" be in a non-identifying relationship with "Projects" instead, which would make them independent from the project's existence, and they wouldn't be connected to the existence of the user or the project.

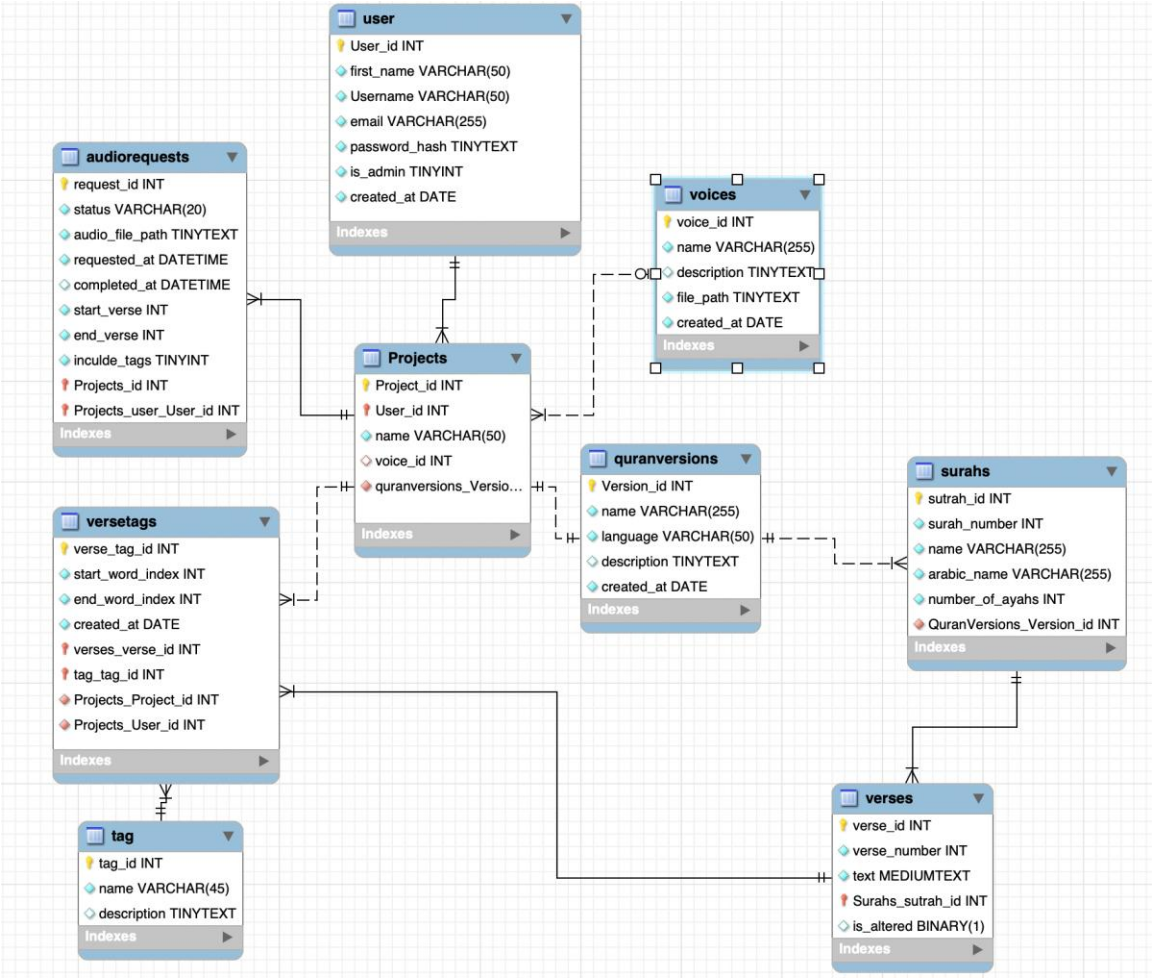

Figure 45: Latest iteration of our database

6.1.4 Website Development

To connect our website's front-end part with the back-end part, we've used Flask, a " lightweight WSGI web application framework" [47], which uses Python to be implemented. Using Flask, we were able to connect our MySQL database to our website's pages: Login, sign up, home page, etc.

Figure 46 shows the __init__.py for the framework. First, it configures the secret key and the database used for our system, then, it initializes the login by Flask, gets the user's ID from the database, debugs the app, and finally launches it.

```python
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager

db = SQLAlchemy()

def create_app():
    app = Flask(__name__)

    # ✅ Set Secret Key and Database Configuration
    app.config['SECRET_KEY'] = 'bqidfbiwfwiucwuicbwiiwnfw'
    app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+mysqlconnector://root:@localhost/mydb'

    db.init_app(app)

    # ✅ Import blueprints
    from .views import views
    from .auth import auth

    # ✅ Fix typo in 'url_prefix'
    app.register_blueprint(views, url_prefix='/')
    app.register_blueprint(auth, url_prefix='/')

    from .modules import User, Project, Quranversions, Voices

    # ✅ Initialize Flask-Login
    login_manager = LoginManager()
    login_manager.login_view = 'auth.sign_in'  # Make sure 'sign_in' is correctly defined in 'auth.py'
    login_manager.init_app(app)

    # ✅ Flask-Login expects ID from get_id()
    @login_manager.user_loader
    def load_user(user_id):
        return User.query.get(int(user_id))

    # ✅ Debugging
    app.config['DEBUG'] = True
    app.config['TEMPLATES_AUTO_RELOAD'] = True

    return app
```

Figure 46: Python code of the starter file __init__.py

6.2 Front-End Development

We've put our HTML, CSS, and JavaScript code-writing to use for this part of the website development. A variety of versions and prototypes were made for the presentation part of the website that is visible to the user. HTML provided the structure, CSS provided the look and visual interactions, and JavaScript provided the behavior of the website and was the direct key to utilizing the Flask framework to connect the database to the website.

Figure 47 shows the first version we built of the login page, while Figure 48 shows the finalized version. The final version has a better, more suitable logo. It's neatly arranged, and an indication is added for the user who hasn't made an account yet to sign up. A tagline and a welcoming phrase were also added to make the website more welcoming and inviting.

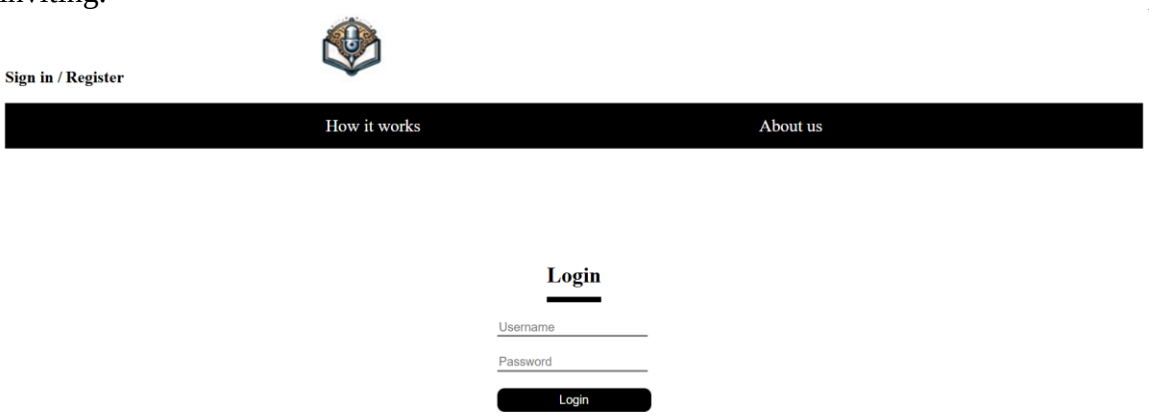

Figure 47: First interactive version of the login page

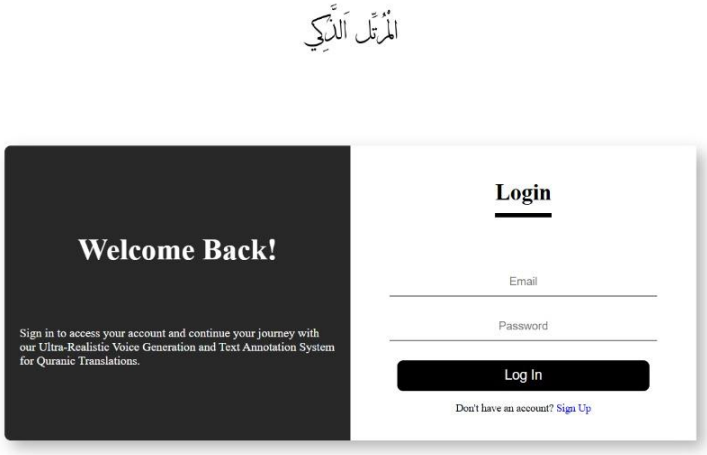

Figure 48: Finalized interactive version of the login page

Figure 49 shows the finalized version of the sign-up page, which also has an indication at the bottom that tells the user to log in instead if they do happen to have an account.

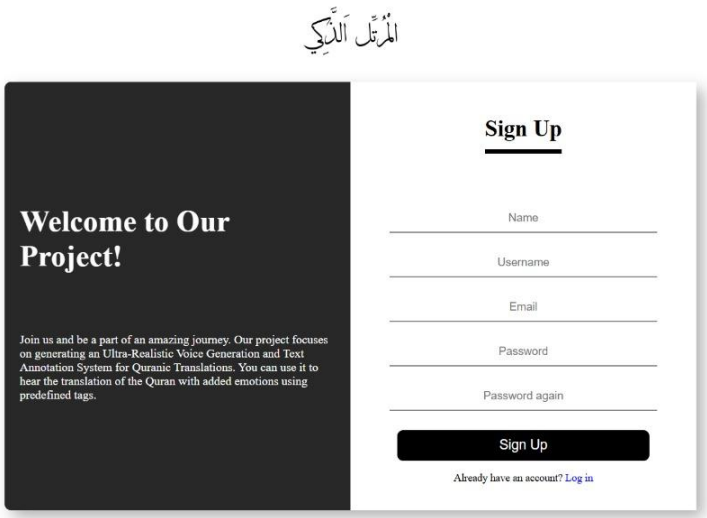

Figure 49: Finalized interactive version of the sign-up page

Figure 50 shows the finalized version of the "About Us" page, which gives the user an introduction to the people behind the making of our system.

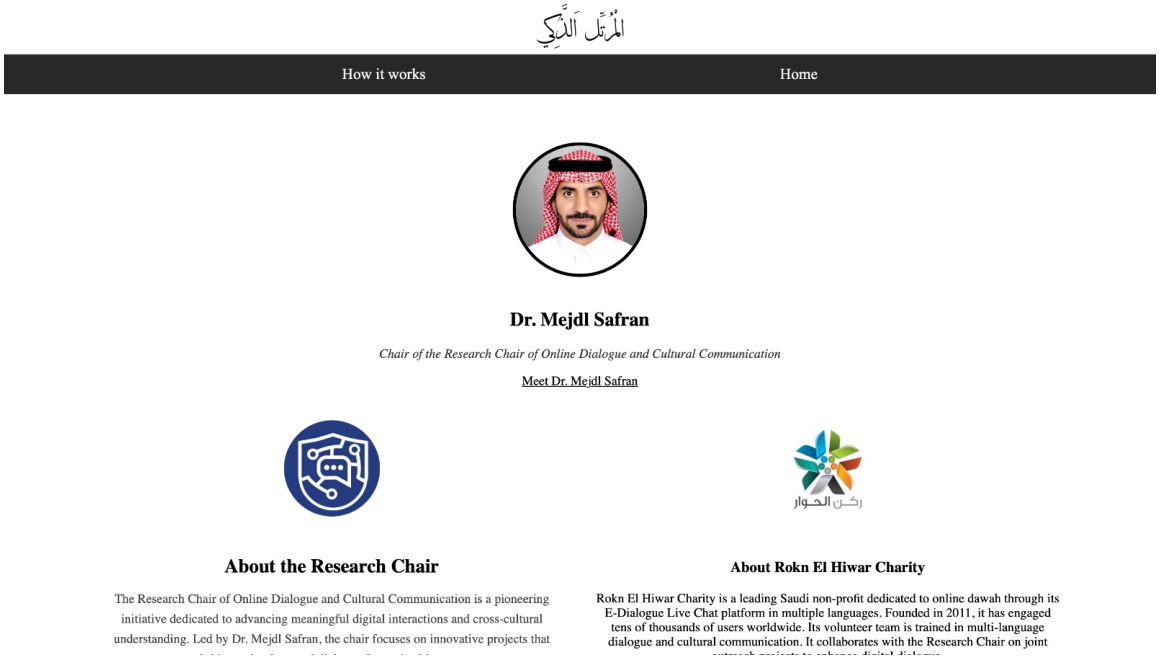

Figure 50: Finalized version of the "About Us" page

Figures 51 and 52 show the finalized version of the "How It Works" page, which educates the user on how to navigate the usage of the system. Figure 51 shows the user how to use the system's website through steps, and figure 52 shows a list of things the user can do in the system while creating the project.

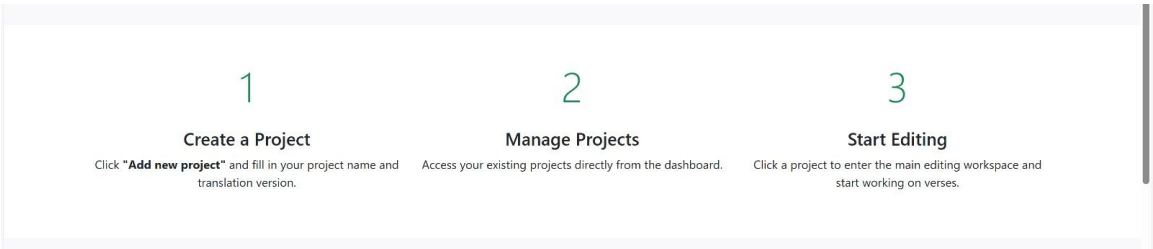

Figure 51: Finalized version of the "How It Works" page, which contains the steps on how to navigate the system's website

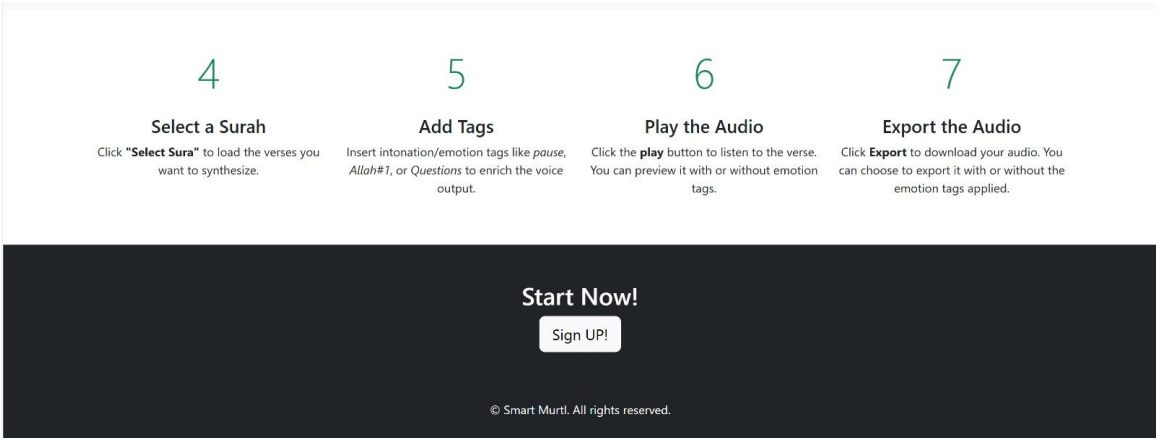

Figure 52: Steps 4-7 contain what the user could do when editing the project

Figure 53 shows a finalized version of the home page, where the user can check how the project works, about us, add their projects or delete them, and choose to edit them.

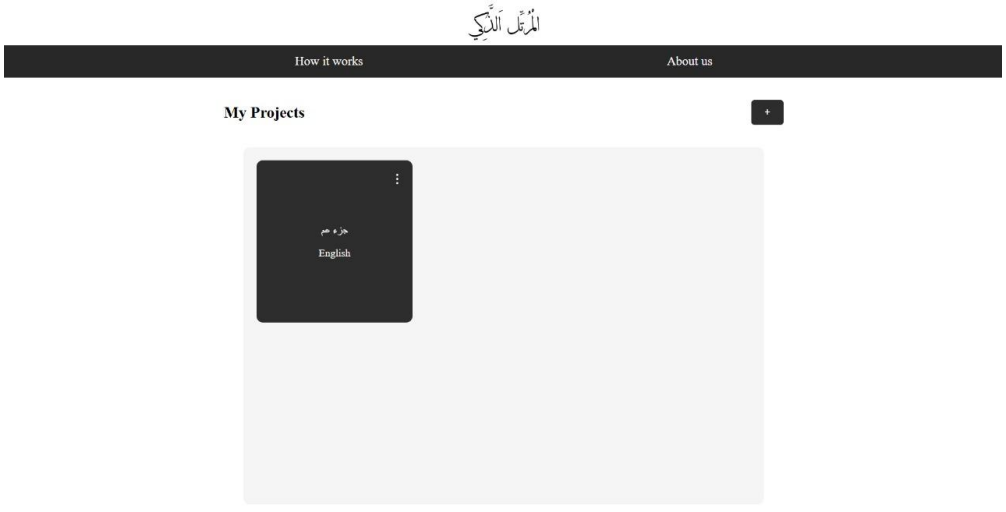

Figure 53: Finalized interactive version of the home page, where projects are added by the signed user

Figure 54 shows the finalized version of the main page, where the user, after choosing to edit a project, can choose the voice, surah, verse, and tags to apply.

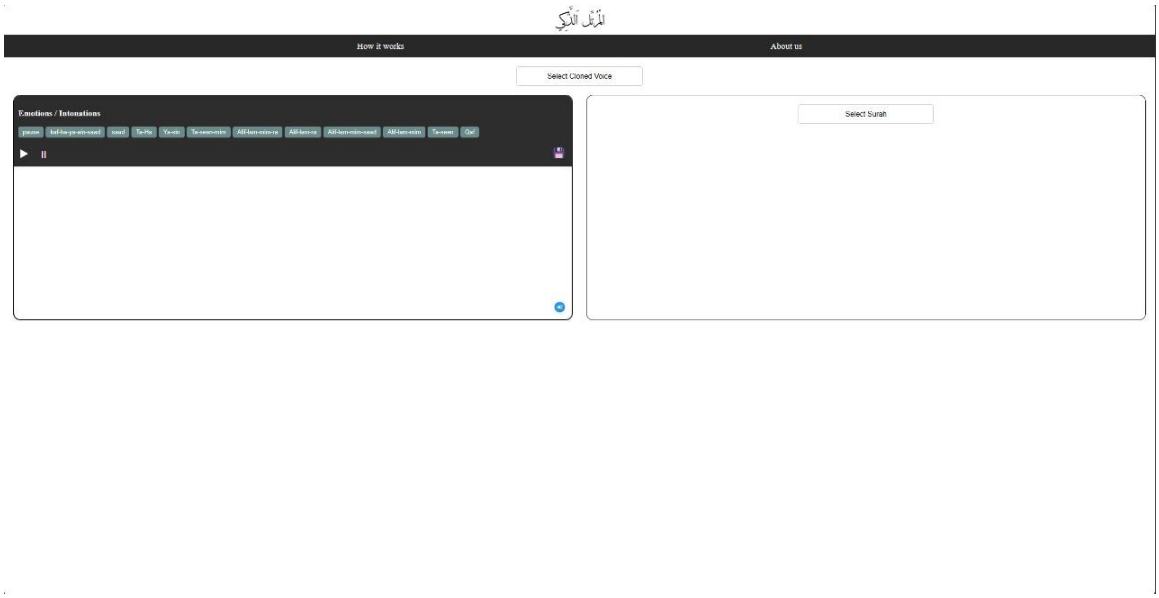

Figure 54: Finalized interactive version of the main page

Figure 55 shows the HTML code for the home page, where the user can check how the project works, about us, and add their projects or delete them. This file is written so that it could be formatted to Flask's accommodations of work, for example, 'static' in the first few lines refers to the file 'static' which our images, CSS files, and JavaScript files for our templates – our HTML files – are stored.

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My Projects</title>
        <link rel="stylesheet" href="{{ url_for('static', filename='css/Home.css') }}">
        <script defer src="{{ url_for('static', filename='js/Home.js') }}"></script>
        <style>
            /* Position delete button in the top-right of the project card */
            .project-card {
                position: relative; /* Required for absolute positioning inside the card */
            }

            .project-card .delete-btn {
                position: absolute;
                top: 10px;
                right: 10px;
                background: none;
                border: none;
                color: white;
                font-size: 20px;
                cursor: pointer;
            }

            .project-card .delete-btn:hover {
                color: red;
            }
        </style>
    </head>
    <body>
        <div class="header">
            <img src="{{ url_for('static', filename='images/logoo.png') }}" alt="Logo">
        </div>
        <main>
            <a href="{{ url_for('views.how_it_works') }}">How it works</a>
            <a href="{{ url_for('views.about_us') }}">About us</a>
        </main>

        <section class="projects-section">
            <div class="projects-header">
                <h2>My Projects</h2>
                <button id="addProjectBtn">Add new project</button>
            </div>

            <div id="projectsGrid" class="projects-grid">
                {% for project in project_data %}
                    <div class="project-card"
                        data-id="{{ project.Project_id }}"
                        data-version-name="{{ project.quranversion.name }}">
                        <!-- ✅ Pass the correct data-id -->
                        <button class="delete-btn" data-id="{{ project.Project_id }}">&times;</button>
                        <p class="project-title">{{ project.name }}</p>
                        <p class="project-lang">Language: {{ project.quranversion.language }}</p>
                        <button class="project-btn"
                        onclick="window.location.href='{{ url_for('views.project_details', project_id=project.Project_id) }}'">
                    Next
                </button>
                    </div>
                {% endfor %}
            </div>
        </section>

        <!-- ✅ Modal Popup for adding projects -->
        <div id="projectModal" class="modal">
            <div class="modal-content">
                <span id="closeModal" class="close">&times;</span>
                <img src="{{ url_for('static', filename='images/logoo.png') }}" alt="Logo" class="modal-logo">
                <form id="projectForm">
                    <input type="text" id="projectName" placeholder="Project Name" required>

                    <!-- ✅ Quran Versions populated dynamically -->
                    <select id="translatedVersion" required>
                        <option value="" disabled selected>Select Translated Version</option>
                        {% for version in quranversions %}
                            <option value="{{ version.name }}">{{ version.name }},{{ version.language }}</option> <!-- ✅ Use name instead of id -->
                        {% endfor %}
                    </select>

                    <select id="language" required>
                        <option value="" disabled selected>Select Language</option>
                        <option value="English">English</option>
                        <option value="Arabic">Arabic</option>
                    </select>
                    <button type="submit">Next</button>
                </form>
            </div>
        </div>

        <!-- ✅ Modal for deletion confirmation -->
        <div id="deleteModal" class="delete-modal">
            <div class="modal-content">
                <span id="closeDeleteModal" class="close">&times;</span>
                <p>Are you sure you want to delete this project?</p>
                <button id="confirmDelete">Yes</button>
                <button id="cancelDelete">No</button>
            </div>
        </div>
    </body>
</html>
```

Figure 55: The HTML code for the home page

Chapter 7: System Testing

This chapter discusses the tests conducted on the system after the behavioral building blocks were laid out. In the following section, we will show the evaluation results of the models we have tested for voice cloning, and the tests we have conducted for validating the functionality of the website components.

7.1 Voice Dataset

The dataset we used for our training was .wav files for verses and surahs from the Quran, some verses contained Arabic characters that are pronounced independently (For instance: الٓمٓ، كٓهيعٓصٓ), and others did not. The voice which the dataset was compiled from was that of Mikael Waters, and his recordings were provided by the Rokn El Hiwar[2] Organization. The number of files reached 203 files with an average of 5 segments per file, totaling 282 files which range from a second to 13 seconds long.

7.2 Evaluation Metrics

The evaluation metrics we used for the three experimental voice cloning models are as follows:

- **Mel Cepstral Distortion (MCD):** is a measure of how different two sequences of mel cepstral are. [48] Lower MCD = Better speech quality.
- **Perceptual Evaluation of Speech Quality (PESQ):** Defined as "result of integration of the perceptual analysis measurement system." [49] It returns a score between -0.5 and 4.5, the higher scores indicating a better quality.
- **Character Error Rate (CER):** measures the rate of erroneous characters produced by synthesized voice compared to the ground truth voice. [50]
- **Word Error Rate (WER):** It's the same as a CER but in words a lower WER means better quality. [50]
- **F0 Contour Deviation (Pitch Analysis):** represents the pitch trajectory over time and is crucial in speech prosody, which conveys intonation, stress, and emotion.
    - Mean Absolute Error (MAE): to measure the difference between predicted and actual acoustic features. Lower MAE indicates that the predicted spectrogram is closer to the real spectrogram, meaning better voice cloning quality. [51]
    - Root Mean Square: to measure the difference between predicted and ground truth spectrograms [52]
    - DTW Alignment Distance: The **lower the DTW alignment distance, the better** the model's ability to match the natural speech prosody. [53]
- **Global Variance (GV) :** represents how much the Global Variance (GV) of the synthesized speech deviates from the reference speech. A lower GV difference

means that the synthesized speech closely matches the natural variance of human speech. [54]

- **Mean Opinion Score (MOS):** "Subjective rating of audio quality on a scale from 1 to 5." [55]
- **Model Speed:** The amount of time (in seconds) it takes for the model to play the generated voice file after pressing the play button.

7.3 Models' Evaluation

In this subsection, we take each of our chosen models being put to the test and measure each one using our evaluation metrics. We've used Notion, a customizable tool used to organize data, to show our results and our evaluations using our evaluation metrics. Figure 56 shows the "Voice Cloning Model Comparison" page we've made using the Notion tool, which can play each model's resulted voice file when voice cloning a Quran verse and shows our results, which can be seen by clicking this link: https://www.notion.so/Voice-Cloning-Model-Comparison-19199bfc2ac18041bb3bd0bcb598a157

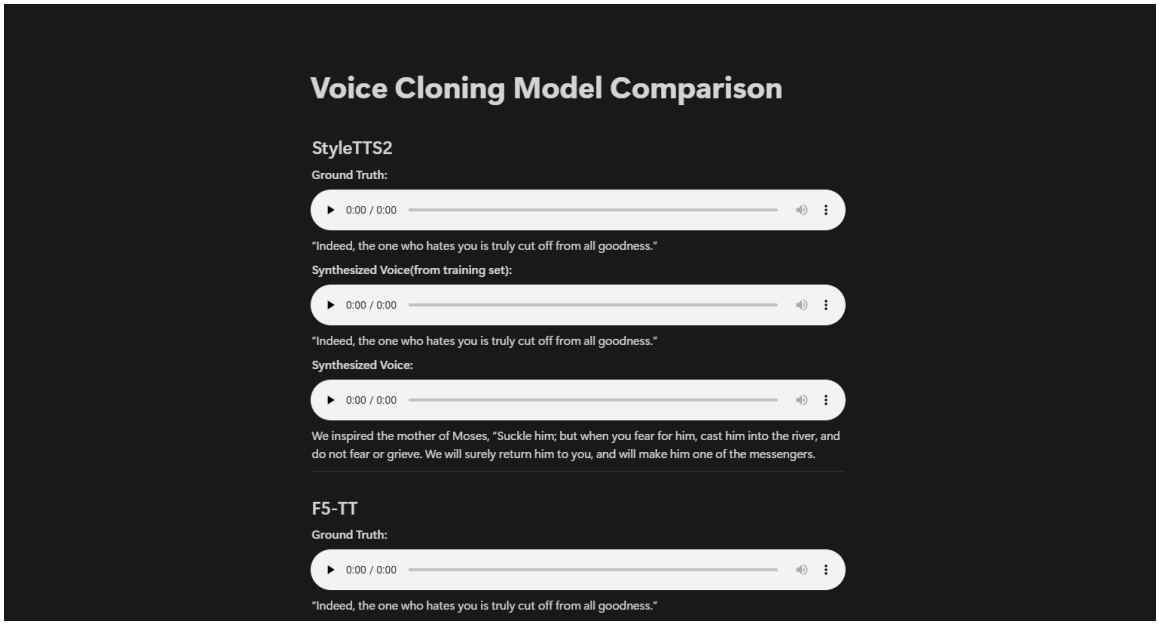

Figure 56: The Notion page, "Voice Cloning Model Comparison"

We'll divide this subsection into two: One time without the use of tags, and another using them.

7.3.1 Without Tagging

Using the standard evaluation metrics for voice cloning models, we were able to determine the efficiency of each model we found and considered. We have divided the dataset into training, validation and testing datasets. We've also generated voice recording from the surahs with the training data – already seen data by the models – and generated voice recording from the surahs in the testing data – which was considered unseen data by the models. Then, we've provided the results for these two types of scenarios using standard voice cloning evaluation metrics.

Table 7 shows each voice cloning model measured in evaluation metrics when using our provided training set, and in Table 8, we measure the models again after using our test set instead. There are notable differences between the two sets of measurements, for example: The low MCD that we measured from StyleTTS2 when the training set was used indicates that the speech quality was better than when the test set was used, which had a high MCD. However, the character error rate (CER) for Tortoise-TTS is higher when the training set is used, and it's the only model to have a high percentage of CER while the others have 0%, highlighting its inefficiency compared to the other two models. So, ultimately, StyleTTS2 was chosen out of the three models, and we used two versions for it, one without tagging, and one with tags added in.

Table 7: Evaluation metrics table (from training set)

Model Name	Mel Cepstral Distortion (MCD)	Perceptual Evaluation of Speech Quality (PESQ)	Character Error Rate (CER)	Word Error Rate (WER)	Mean Absolute Error (MAE)	Root Mean Square Error(RMSE)	DTW Alignment Distance	Global Variance (GV)	Mean Opinion Score (MOS)
StyleTTS2	19.269341969875184	1.159222936630249	0.00%	0.00%	3.42	10.74	537.69	302.7468	4.3
F5-TTS	17.571161929761526	1.2934679985046387	0.00%	0.00%	8.83	22.21	2508.30	88.5972	4.36
Tortoise-TTS	9.731695808468078	1.0664148330688477	52.31%	76.92%	7.82	25.44	3856.09	740.4001	

Table 8: Evaluation metrics table (Test set)

Model Name	Mel Cepstral Distortion (MCD)	Perceptual Evaluation of Speech Quality (PESQ)	Character Error Rate (CER)	Word Error Rate (WER)	Mean Absolute Error (MAE)	Root Mean Square Error(RMSE)	DTW Alignment Distance	Global Variance (GV)	Mean Opinion Score (MOS)
StyleTTS2	22.38814076172558	1.0328679084777832	0.00%	0.00%	18.68	37.92	8610.41	732.3813	4.3
F5-TTS	19.9179740136363O4	1.1552864313125 61	0.00%	0.00%	23.80	43.49	16089.98	565.4701	4.36
Tortoise-TTS	8.542806217640917	1.1290837526321411	40.40%	52.50%	17.24	42.25	14429.39	451.7038	

Table 9 shows the normal speed in seconds it takes each model to generate the voice file for the selected verse when the play button is pressed.

Table 9: Speed evaluation table

	Speed in seconds
StyleTTS2	2 - 5
F5-TTS	4 -12
Tortoise-TTS	20

7.3.2 With Tagging

After selecting StyleTTS2 as our final model, we conducted a simple comparison between two versions of the model. We saw the first version, which was trained without any special tags (e.g., <pause>, <Ta Ha>). The second version was trained with these tags embedded in the input text.

Table 10 shows the two versions of StyleTTS2 measured in evaluation metrics when using our training set. And table 11 shows the two versions measured in the same evaluation metrics, but with the test set used instead. The character error rate (CER) percentage is noticeably lower when the test set is used for both versions. However, there's a minimal change in the root mean square error (RMSE) between the usage of the two sets.

Table 10: Evaluation metrics (training set) with and without tags of StyleTTS2

Model Name	Mel Cepstral Distortion (MCD)	Perceptual Evaluation of Speech Quality (PESQ)	Character Error Rate (CER)	Word Error Rate (WER)	Mean Absolute Error (MAE)	Root Mean Square Error (RMSE)	DTW Alignment Distance	Global Variance (GV)	Mean Opinion Score (MOS)
With Tags	15.984047958245132	1.208592295646667	5.45%	6.26%	17.202	22.2045097351	8610.41	258.9804	4.01
Without Tags	19.44127610202335	1.118824958801270	14.09%	27.91%	16.183	22.204509735107	80725.66	155.4935	4.2

Table 11: Evaluation metrics (test set) with and without tags of StyleTTS2

Model Name	Mel Cepstral Distortion (MCD)	Perceptual Evaluation of Speech Quality (PESQ)	Character Error Rate (CER)	Word Error Rate (WER)	Mean Absolute Error (MAE)	Root Mean Square Error (RMSE)	DTW Alignment Distance	Global Variance (GV)	Mean Opinion Score (MOS)
With Tags	16.989619036146607	1.287939071655273	1.04%	5.26%	17.525	22.970333099365	47226.41	261.1662	4.01
Without Tags	19.73001752924427	2.426105022430420	4.15%	18.42%	17.371	21.815542221069	71732.89	159.3708	4.2

7.4 Dashboard Testing

In this section, we show tests that were held for the website after we've connected the website to the database and its database was established and implemented; it's necessary to conduct these tests to determine the functionality, adaptability and availability of the website.

Tests in Tables 12, 13, and 14 were all conducted to test the functionality of the login page. Table 12 shows our test that login would not be allowed if there were no credentials entered.

Table 12: First dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Login.	No credentials.	No sign in, and an error message is printed.	No sign in, and "Invalid email or password" message is printed.

Table 13 shows that a login would not be allowed if entered credentials were either invalid or not previously signed up in the database.

Table 13: Second dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Login.	Invalid/non-signed credentials.	No sign in, and an error message is printed.	No sign in, and "Invalid email or password" message is printed.

Table 14 shows that a login would be successful if the credentials entered were correct and was previously signed up.

Table 14: Third dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Login.	Correct/already signed up credentials	Should sign in successfully	Signed in successfully

The next group of tables show tests that were conducted to test the functionality of the sign-up page. Table 15 shows that credentials entered in the sign-up form that weren't signed up before should create a new account with those credentials.

Table 15: Fourth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Sign-up.	New credentials	Should successfully create a new account.	A new account was created successfully

Table 16 shows that signing up would not be allowed if entered credentials were signed up previously.

Table 16: Fifth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Sign-up.	Already signed credentials	Should not create a new account, and an error message appears instead.	No account was created, and an error message is printed.

Table 17 shows that an account would not be created if the requested password's length was shorter than 8 characters.

Table 17: Sixth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Sign-up.	Password length < 8	An error message should appear saying that the length of the password should be 8 characters or higher.	Said error message appeared.

For the next group of tables, we show tests that were conducted to test the functionality of the home page. Table 18 shows that if the user was successful in logging in, the projects that were already created by them should appear in the "My Projects" section.

Table 18: Seventh dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Home page	Saved projects	When login is successful, the projects created by that user should appear in "My Projects" square.	The logged in user's projects appeared in the "My Projects" square.

Table 19 shows that when a user adds a project, a form should appear that requests the user to enter the project's name, choose the translated version of the Quran the user prefers out of a list, and choose the language that the user prefers out of a list.

Table 19: Eighth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Home page	Add a project	A form should appear for the user, where they name the project, choose the translated version, and choose the language. Afterwards, it appears in "My Projects" square.	Said form appeared after pressing "Add new project", and the project was added into said square after filling said form.

Table 20 shows that when a user deletes a project, a warning should appear that asks the user if they're sure they want to delete the project or not. If the user presses "Yes", the project should be deleted.

Table 20: Ninth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Home page	Delete a project	A yes or no warning should appear asking the user if they're sure they want to delete the project. And if the user presses "yes", it should be deleted.	Said warning appeared, and when "yes" was pressed, the project was deleted.

Tables 21 to 25 show tests conducted on the main page to test the core functionality of the system. Table 21 shows that when a user selects a surah, then presses the play button, it shouldn't play anything since the verse was not selected, and therefore a voice file can’t be generated.

Table 21: Tenth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Main page	Pressing the play button without selecting a verse after a surah is selected.	There should be no generated voice files, and an alert should appear telling the user that they have to select a verse first before pressing the play button	An alert saying, " Please select a verse first!" appeared, and no voice file was generated.

Table 22 shows that when a user selects a surah, then selects a verse, then presses the play button, it should generate the desired voice file which reads the chosen verse.

Table 22: Eleventh dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Main page	Pressing the play button after selecting a verse inside a surah.	A voice file of the chosen verse should be generated seconds after the play button is pressed.	The generated voice file which reads the verse was played after the play button was pressed.

Table 23 shows that when a user presses the export button after generating a voice file, a window should appear asking the user where to store the generated voice file.

Table 23: Twelfth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Main page	Export a generated voice file of a verse.	The export window should appear and ask the user where they'd prefer to store the exported file.	The export window appeared, and the file was exported.

Table 24 shows that when a user chooses a surah that includes a selection of tags to be optionally used, if the user chooses to tag the verse, the added tags should be highlighted in their proper place on the verse, and the generated file should be altered accordingly.

Table 24: Thirteenth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Main page	Add tags to a verse.	The verse should have different colored highlights mapped according to the available tags for the verse, and the generated voice file should sound altered according to the tags.	The verse was highlighted in various colors indicating each tag's place on the verse, and the tags changed how the generated file sounded.

Table 25 shows that when a user chooses to remove the added tags from the verse, the highlights should be gone, and they shouldn't affect the generation of the file.

Table 25: Fourteenth dashboard test

Test Subject	Test Case	Expected Output	Actual Output
Main page	Remove tags from a verse.	The highlights on the verse should be gone, and the generated file should be generated the normal way.	The highlights disappeared, and the generated voice file was generated as normal as it was without the tags.

Chapter 8: Summary

There is a growing need for natural, high-quality speech synthesis for Quranic translations, especially in non-Arabic languages like English. Current systems for voice cloning and TTS often struggle with achieving naturalness, emotional tone, and intelligibility in a religious context, where precision and clarity are crucial.

We proposed using Tacotron2 for voice cloning, as it offers high-quality speech synthesis with naturalness and intelligibility and uses an encoder-decoder architecture with attention mechanisms, enabling it to produce smooth, accurate speech even for difficult pronunciations. For text annotation, we utilized spaCy, a powerful NLP library that combines rule-based and deep learning models. This approach provided a flexible pipeline for processing text, improving the accuracy of semantic annotation.

In the next phase of the project, we focused not only on training the Tacotron2 model with a carefully curated dataset of voices and corresponding transcripts, but we also tried to test other models: StyleTTS2, F5-TTS, and Tortoise-TTS, to see better, more convenient options. With the final declared model being StyleTTS2. We will also fine-tune spaCy's pipeline to improve annotation accuracy for domain-specific texts. Additionally, we experimented with other voice cloning and text annotation techniques to further improve performance. Alongside these technical developments, we implemented a system that allows users to interact with the model, exploring its integration into real-world applications and gathering feedback for continuous improvement.

We also implemented our website where our page prototypes and models were deployed. When a user signs in, he's able to view the different projects he created on the home page, add new projects, edit, or delete existing projects. Also, on the main page when the user chooses to edit a certain project, he's able to choose a voice, choose a sura – where its look depends on the version of Quran they chose, and have the freedom whether to use one or more of the available tags for the chosen surah and verses or not before exporting the generated voice file.

References

[1] Alghamdi, Mansour, Chafic Mokbel, and Mohamed Mrayati. "Arabic language resources and tools for speech and natural language." *Proceedings of the 2nd International Conference on Arabic Language Resources & Tools, Cairo*. 2009. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=22a25549c2d47c257f0e6eb2d794b19d38ee934a

[2] Rukn Al-Hewar, "Rukn Al-Hewar Non-profit Organization." https://edialoguec.org.sa/

[3] Tife Sanusi "Everything you need to know about voice cloning" Deepgram https://deepgram.com/learn/voice-cloning-everything-to-know

[4] Ruggiero, Giuseppe, et al. "Voice cloning: a multi-speaker text-to-speech synthesis approach based on transfer learning." *arXiv preprint arXiv:2102.05630* (2021). https://arxiv.org/abs/2102.05630

[5] Wadoux et al. "Voice Cloning for Voice Disorders: Impact of Phonetic Content." In *International Conference on Text, Speech, and Dialogue*, pp. 293-303. Cham: Springer Nature Switzerland, 2023. https://dl.acm.org/doi/10.1007/978-3-031-40498-6_26

[6] "What is a Neural Network?" AWS Amazon https://aws.amazon.com/what-is/neural-network/ Accessed 5 Oct, 2024.

[7] "What is machine learning?" IBM Cloud Education What Is Machine Learning (ML)? | IBM Accessed 5 Oct, 2024.

[8] "image of neural network" Science learning Hup, 14 Mar 2023 https://www.sciencelearn.org.nz/images/5156-neural-network-diagram

[9] Mehdi Lotfinejad. "Most Common Deep Learning Applications" Dataquest, 10 Mar, 2023 https://www.dataquest.io/blog/6-most-common-deep-learning-applications/

[10] Neekhara et al. "Expressive neural voice cloning." In *Asian Conference on Machine Learning*, pp. 252-267. PMLR, 2021. https://proceedings.mlr.press/v157/neekhara21a.html

[11] "What is Voice Cloning?" Elevenlaps https://elevenlabs.io/blog/what-is-voice-cloning

[12] Martin Thissen "Voice Cloning for Beginners In 8 Minutes" Medium, 16 Nov, 2022 https://medium.com/@martin-thissen/voice-cloning-for-beginners-in-8-minutes-db046e009a70

[13] Bhattacharjee, Soumyadeep. "Deep learning for voice cloning." Master's thesis, 2021. https://elib.uni-stuttgart.de/bitstream/11682/11721/1/Bhattacharjee_Master_Thesis.pdf

[14] "Text annotation for machine learning" Superannotate, 27 Nov, 2023 https://www.superannotate.com/blog/text-annotation-for-machine-learning

[15] J. Shen *et al*., "Natural TTS Synthesis by Conditioning Wavenet on MEL Spectrogram Predictions," *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Calgary, AB, Canada, 2018, pp. 4779-4783, doi:10.1109/ICASSP.2018.8461368

[16] Li, Naihan, et al. "Close to human quality TTS with transformer." *arXiv preprint arXiv:1809.08895* 2 (2018).

[17] Arik, Sercan, et al. "Neural voice cloning with a few samples." *Advances in neural information processing systems* 31 (2018).https://proceedings.neurips.cc/paper_files/paper/2018/file/4559912e7a94a9c32b09d894f2bc3c82-Paper.pdf

[18] Neekhara, Paarth, et al. "Expressive neural voice cloning." *Asian Conference on Machine Learning*. PMLR, 2021. https://proceedings.mlr.press/v157/neekhara21a.html

[19] Chen, Qi, et al. "V2C: Visual voice cloning." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022. https://doi.org/10.48550/arXiv.2111.12890

[20] Ren, Yi, et al. "Fastspeech: Fast, robust and controllable text to speech." *Advances in neural information processing systems* 32 (2019), doi: https://doi.org/10.48550/arXiv.1905.09263

[21] Prenger, Ryan, Rafael Valle, and Bryan Catanzaro. "Waveglow: A flow-based generative network for speech synthesis." *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019.doi: 10.1109/ICASSP.2019.8683143

[22] Lin, Yupian, et al. "DoTAT: A domain-oriented text annotation tool." Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. 2022. https://aclanthology.org/2022.acl-demo.1

[23] Islamaj, Rezarta, et al. "TeamTat: a collaborative text annotation tool." *Nucleic acids research* 48.W1 (2020): W5-W11. https://doi.org/10.1093/nar/gkaa333

[24] Li, Yanzeng, et al. "FITAnnotator: A flexible and intelligent text annotation system." *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Demonstrations*. 2021. https://aclanthology.org/2021.naacl-demos.5

[25] Muhammad, A, Muhammad, S. "AI Voice Generation." Resemble AI, https://www.resemble.ai.(2019)

[26] Weitzman, C, Weitzman, T. "Text to Speech for All Your Devices." Speechify, https://www.speechify.com. (2017)

[27] Mason, A. "Text to speech." Descript, https://www.descript.com. (2017)

[28] Anh, M, Gölge, E, Chen, K, Ebata, T. TensorSpeech/TensorflowTTS. GitHub, (2020),https://github.com/TensorSpeech/TensorFlowTTS.git

[29] Rhasspy Community. Rhasspy/Piper. GitHub. (2023). https://github.com/rhasspy/piper.

[30] SVC Develop Team. svc-develop-team/so-vits-svc, GitHub. (2023). https://github.com/svc-develop-team/so-vits-svc

[31] Sharma, M, Rasmuson, D, Rieger, B. “Text Annotation & Labeling Tool ”, LabelBox, https://www.labelbox.com.(2018)

[32] Explosion AI. "spaCy: Industrial-Strength NLP". spaCy, https://spacy.io. (2024)

[33] Akbik, A., Blythe, D., Vollgraf, R. "Flair: An Easy-to-Use Framework for State-of-the-Art NLP". GitHub, https://flairnlp.github.io. (2019)

[34] Qi, P., Zhang, Y., Zhang, Y., Bolton, J., Manning, C. D. "Stanza: A Python NLP Library for Many Human Languages". Stanford NLP Group, https://stanfordnlp.github.io/stanza. (2020)

[35] King Saud university, “Use Case Model Use case diagram.” Accessed: Nov. 27, 2023. [Online]. Available: https://faculty.ksu.edu.sa/sites/default/files/ct1414_lecture3.pdf

[36] Margaret Rouse, “What is the class diagram” Techopedia”, September. 16, 2013. [Online]. Available: https://www.techopedia.com/definition/16466/class-diagram

[37] J. Shen *et al*., "Natural TTS Synthesis by Conditioning Wavenet on MEL Spectrogram Predictions," *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Calgary, AB, Canada, 2018, pp. 4779-4783, doi:10.1109/ICASSP.2018.8461368

[38] Aziz, Azrul Fahmi Abdul, et al. "End to end text to speech synthesis for Malay language using Tacotron and Tacotron 2." *International Journal of Advanced Computer Science and Applications* 14.6 (2023). https://www.proquest.com/scholarly-journals/end-text-speech-synthesis-malay-language-using/docview/2843254733/se-2?accountid=44936

[39] StyleTTS 2: Towards Human-Level Text-to-Speech through Style Diffusion and Adversarial Training with Large Speech Language Models, GitHub. https://github.com/yl4579/StyleTTS2/tree/main

[40] *F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching*. Yushen Chen, Zhikang Niu, Ziyang Ma, Keqi Deng, Chunhui Wang, Jian Zhao, Kai Yu, Xie Chen, Shanghai Jiao Tong University, University of Cambridge, Geely Automobile Research Institute (Ningbo) Company Ltd. https://arxiv.org/pdf/2410.06885

[41] James Skelton, "Choosing the Best Text-to-Speech Models: F5-TTS, Kokoro, SparkTTS, and Sesame CSM", DigitalOcean. https://www.digitalocean.com/community/tutorials/best-text-to-speech-models#sesame-csm

[42] James Betker, *Better Speech Synthesis Through Scaling* https://arxiv.org/pdf/2305.07243

[43] Explosion AI. "spaCy 101: An Introduction to spaCy." spaCy.io, Explosion, https://spacy.io/usage/spacy-101. (2024)

[44] Kwon, Ohsung, et al. "Emotional speech synthesis based on style embedded Tacotron2 framework." *2019 34th International Technical Conference on Circuits/Systems, Computers and Communications 2019.* https://ieeexplore.ieee.org/abstract/document/8793393

[45] F5-TTS: Emotion Driven Text-to-Speech Generator, MimicPC. https://www.mimicpc.com/learn/f5tts-emotion-driven-text-to-speech-generator

[46] TorToiSe, GitHub. https://github.com/neonbjb/tortoise-tts/blob/main/README.md

[47] Flask documentation (3.1.x), Flask. https://flask.palletsprojects.com/en/stable/

[48] Mel Cepstral Distortion, Matt Shannon, GitHub. https://github.com/MattShannon/mcd

[49] *Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs*, *IEEE Xplore* https://ieeexplore.ieee.org/document/941023

[50] WER, CER, and MER. Testing with Kolena. https://docs.kolena.com/metrics/wer-cer-mer/#:~:text=Character%20Error%20Rate%20is%20another%20metric%20that%20measures,are%20useful%20in%20surfacing%20mispronunciations%20and%20erroneous%20phonemes.

[51] Understanding Mean Absolute Error (MAE) in Regression: A Practical Guide, M. Waqar Ahmed, Medium. https://medium.com/@m.waqar.ahmed/understanding-mean-absolute-error-mae-in-regression-a-practical-guide-26e80ebb97df

[52] Root Mean Square Error (RMSE), Jim Frost, Statistics By Jim. https://statisticsbyjim.com/regression/root-mean-square-error-rmse/

[53] What is Dynamic Time Warping (DTW) And How is It Applied in Audio Matching?, milvus. https://milvus.io/ai-quick-reference/what-is-dynamic-time-warping-dtw-and-how-is-it-applied-in-audio-matching

[54] *Incorporating Global Variance in the Training Phase of GMM-based Voice Conversion*, Hsin-Te Hwang, Yu Tsao, Hsin-Min Wang, Yih-Ru Wang, Sin-Horng Chen, Dept. of Electrical and Computer Engineering, National Chiao Tung University, Hsinchu, Taiwan. https://www.researchgate.net/publication/261208295_Incorporating_global_variance_in_the_training_phase_of_GMM-based_voice_conversion

[55] Mean Opinion Score (MOS), VOIP Quality Monitoring Basics, viopfuture. https://www.voipfuture.com/wp-content/uploads/2022/02/VPF123_WP_MOS-Calculation-And-Aggregation.pdf